密码学里的魔法
零知识证明与安全多方计算

刘逸

暨南大学网络空间安全学院

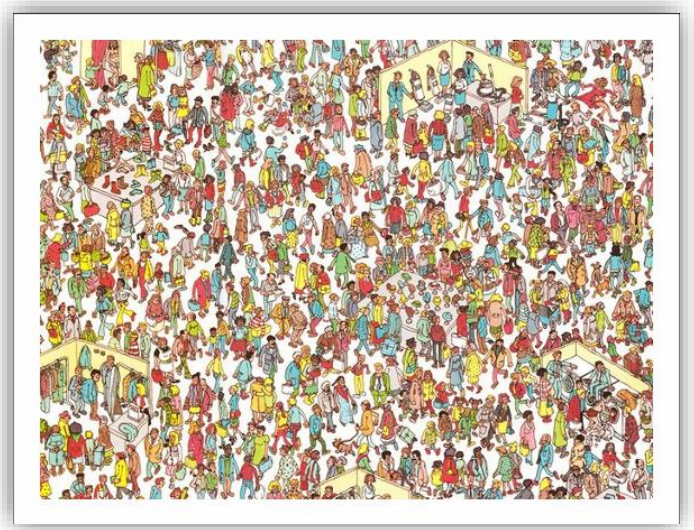
提纲

- 零知识证明（安全多方计算的特例）
  - 什么是零知识证明？
  - 如何实现零知识证明？
- 什么是安全多方计算？
- 如何实现一般的两方计算？
  - 混淆电路 + 不经意传输

零知识证明

场景: Waldo在哪?

Alice



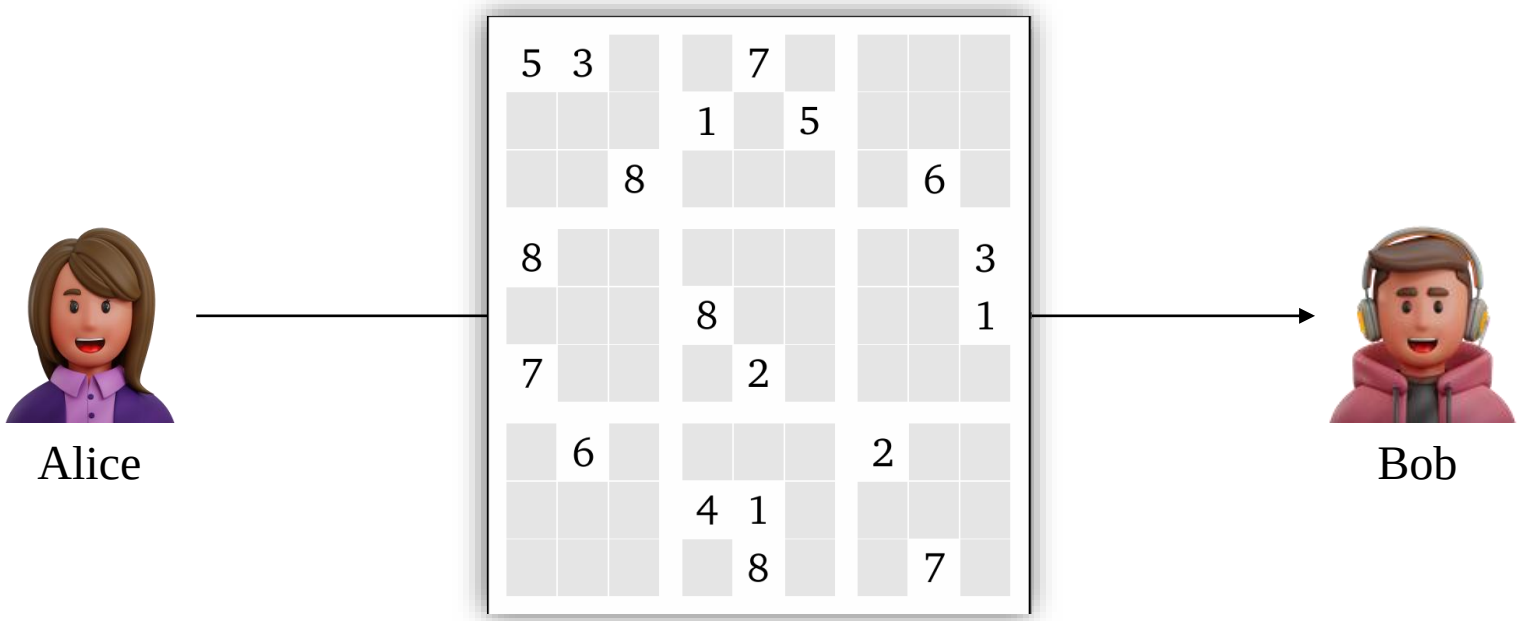
Bob

A “嗨 Bob，我找到Waldo了！”

B “这太快了吧，我不相信。”

场景: 数独

数独的目标是将一个9×9的网格填写上数字，使得每一列、每一行以及每一个3×3的区块内都包含1到9的数字。

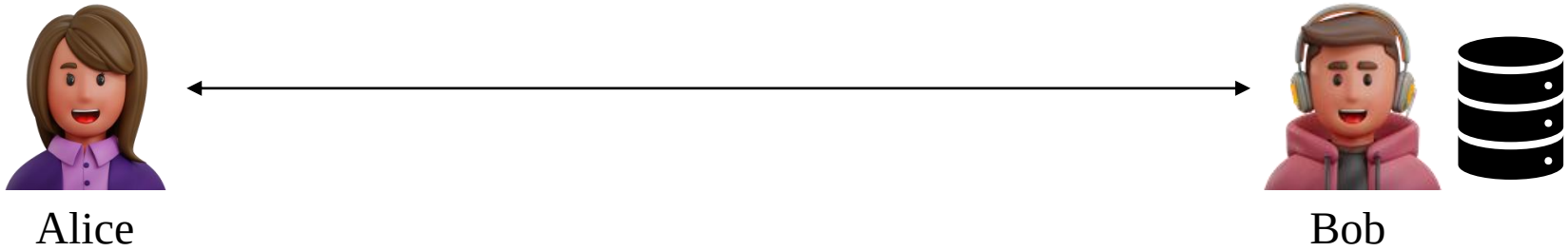


A “嗨 Bob，试试看这个数独题目!”

B “上周你给我一个没有解的数独题，我浪费了3个小时。”

A “这个有解，相信我。”

场景: 认证



A “我能够访问数据库吗？我是Alice。”

B “好的，把密码发给我，让我知道你确实是Alice。”

关于信任和信息的问题

Alice 想要说服 Bob 某些事

- Waldo 在画里面
- 数独题目有解
- Alice 不是被冒名顶替

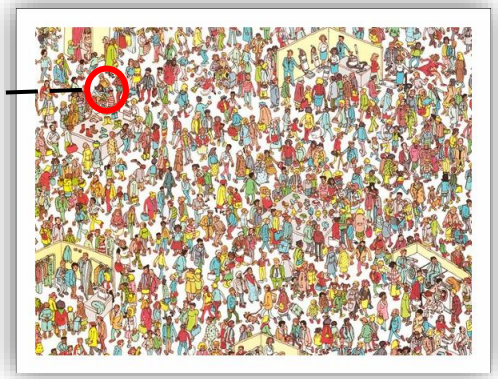
Bob 不应该“过多”了解的信息

- Waldo’s 的位置
- 数独的解
- Alice’的密码

可能的解决方案是什么？

Waldo在哪里? 解决方案

Alice

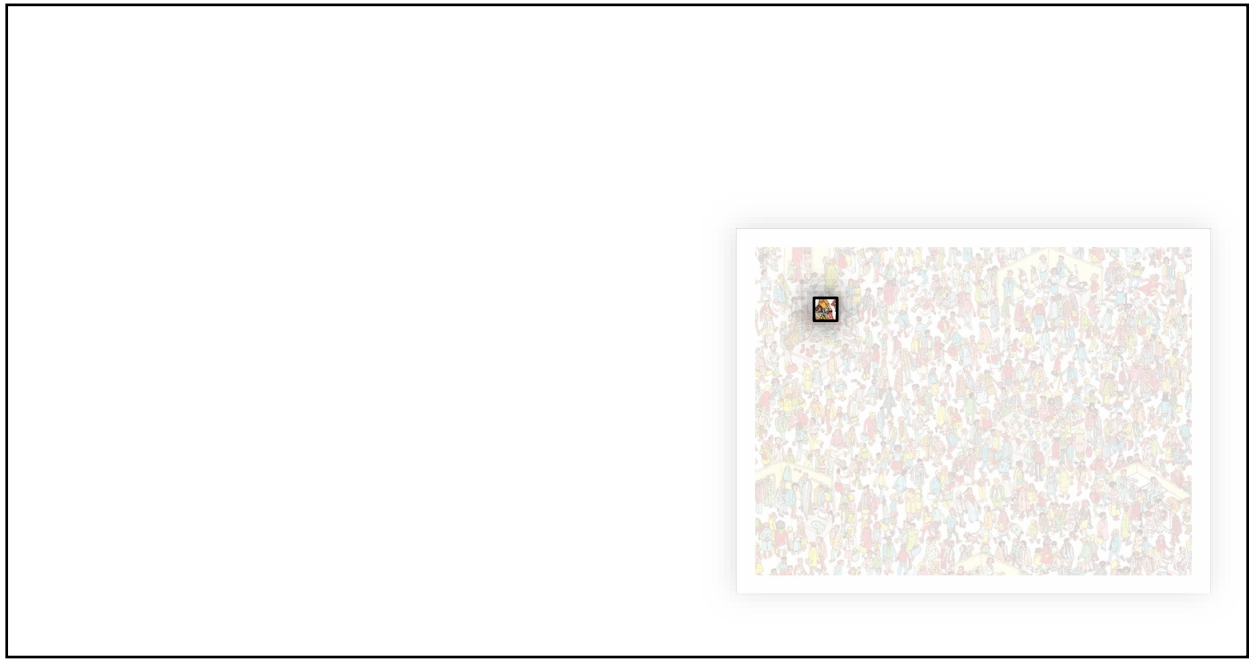


Bob

解决方案

Waldo在哪里? 解决方案

Alice



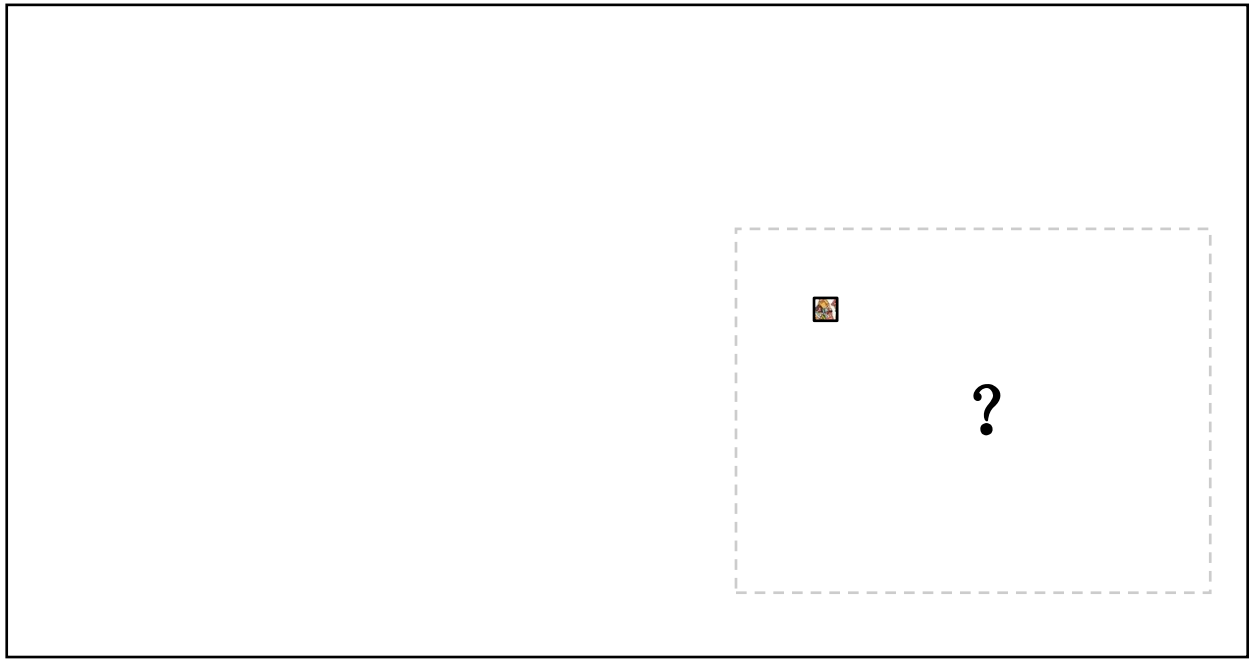
Bob

解决方案

1. Alice在图片上覆盖一个带有小孔的不透明纸板，露出Waldo的位置

Waldo在哪里? 解决方案

Alice



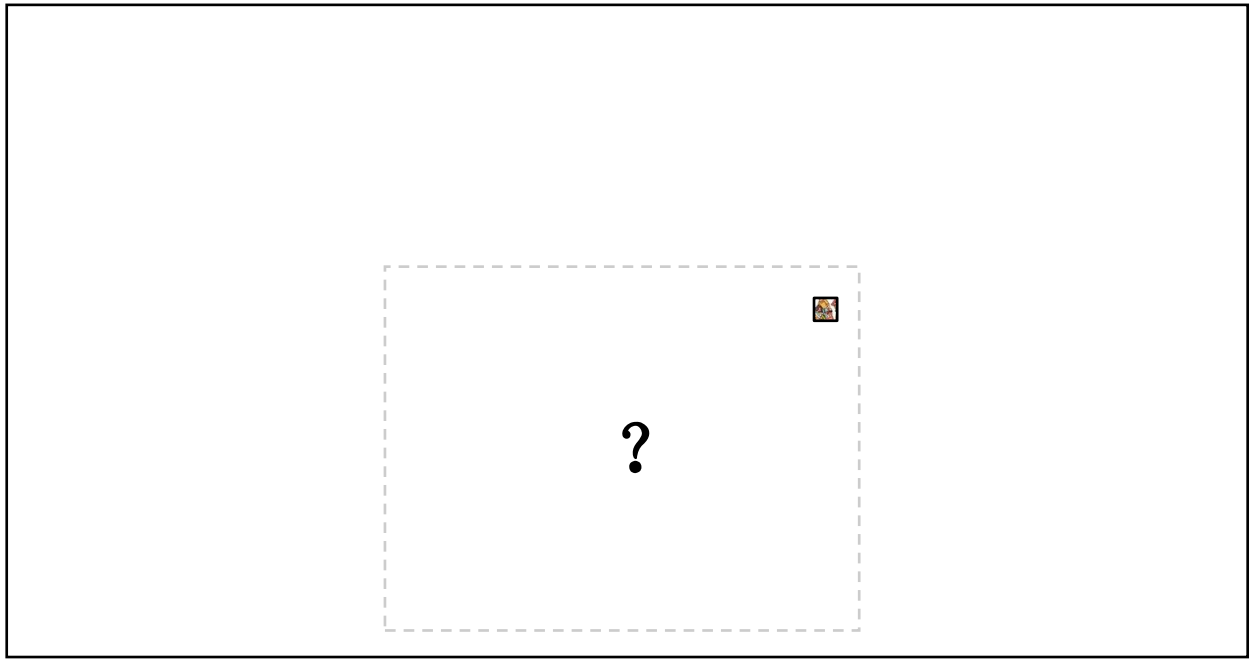
Bob

解决方案

1. Alice在图片上覆盖一个带有小孔的不透明纸板，露出Waldo的位置
Bob 无法获得Waldo在这张图片中的位置信息！

Waldo在哪里? 解决方案

Alice



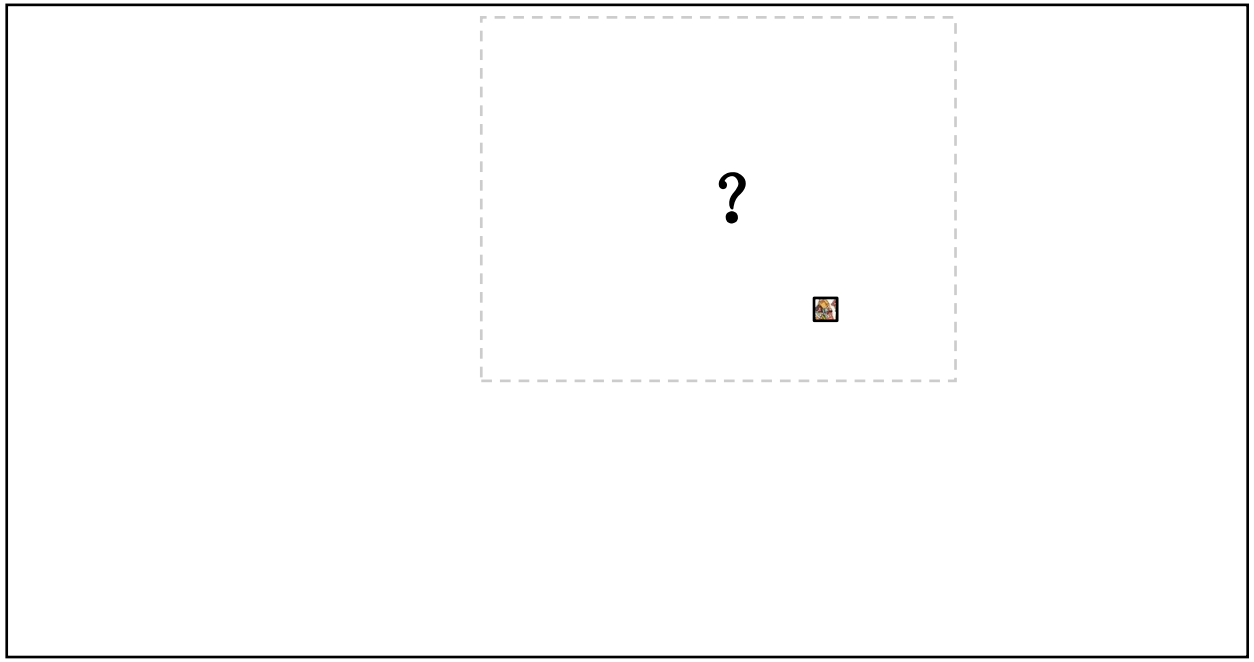
Bob

解决方案

1. Alice在图片上覆盖一个带有小孔的不透明纸板，露出Waldo的位置
Bob 无法获得Waldo在这张图片中的位置信息！

Waldo在哪里? 解决方案

Alice



Bob

解决方案

1. Alice在图片上覆盖一个带有小孔的不透明纸板，露出Waldo的位置
Bob 无法获得Waldo在这张图片中的位置信息！

直观理解

模糊定义

零知识证明 是一种 “不泄露任何额外信息/知识”的前提下，证明并让他人信服某个事实的方法

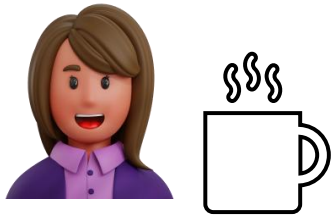
什么是

- 证明某件事?
- 泄露信息/知识?

经典定义中的证明

证明是一系列逻辑步骤的集合。也就是说，Alice可以将其写下来并发送给Bob。

女士品茶



Muriel

Ronald

真实故事 [R. Fisher, *Mathematics of a Lady Testing Tea*, 1956]:

M “先倒茶再倒牛奶，和先倒牛奶再倒茶，味道是不同的。”

R “有意思。你能证明吗?”

M “证明??”

Fisher的巧妙方法: 交互式证明

Muriel

Ronald

随机挑战 私下抛硬币决定冲泡顺序 (先加茶还是牛奶). 把杯子给 Muriel.

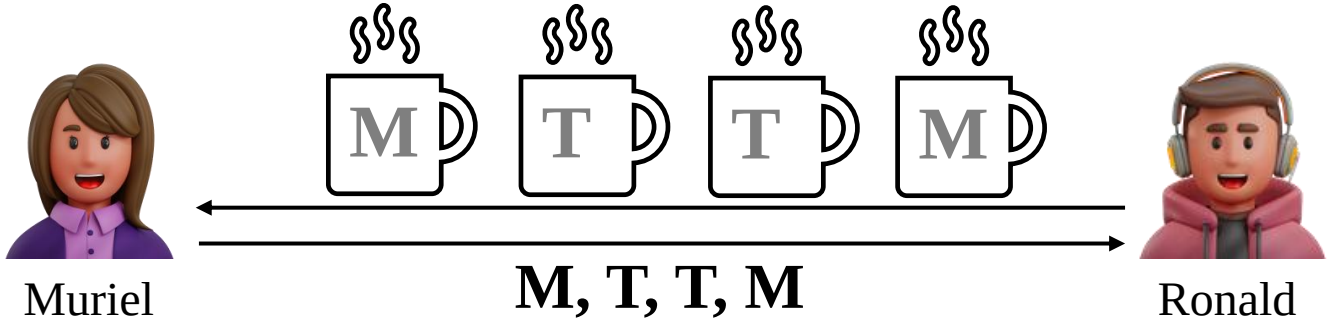
Fisher的巧妙方法: 交互式证明

随机挑战 私下抛硬币决定冲泡顺序 (先加茶还是牛奶). 把杯子给 Muriel.

回应 Muriel 进行猜测.

- 若 Muriel 真能辨别, 则会猜对.
- 若两种冲泡方式无差异, 她猜对的概率为 1/2 .

Fisher的巧妙方法: 交互式证明



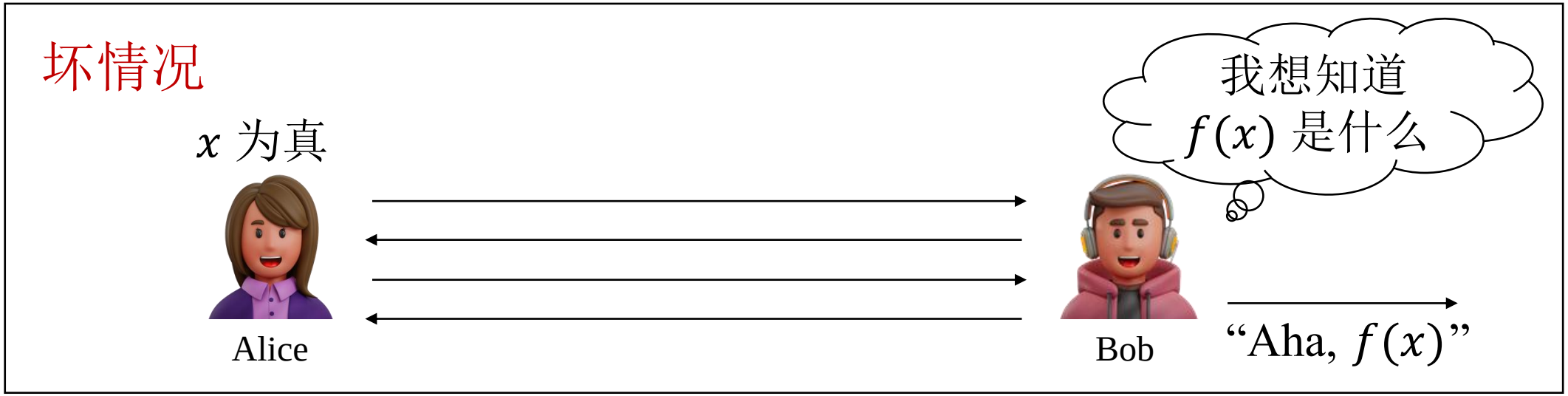
随机挑战 私下抛硬币决定冲泡顺序 (先加茶还是牛奶). 把杯子给 Muriel.

回应 Muriel 进行猜测.

- 若 Muriel 真能辨别, 则会猜对.
- 若两种冲泡方式无差异, 她猜对的概率为 1/2 .

重复 重复 n 次. 若两种方式无差异, 她全部猜对的概率仅为 $(1/2)^n$

认识论: 什么是知识?



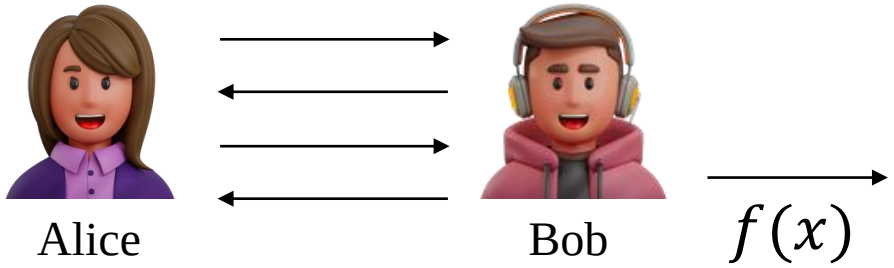
- 如果Bob在交互前无法计算出 $f(x)$，情况就糟了
- 交互记录给了他计算能力

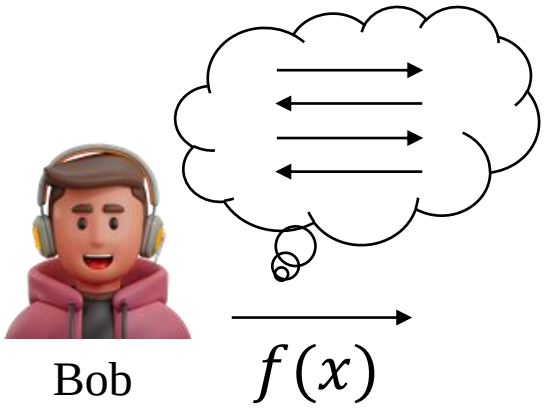
想表达的是:
Bob在看到交互记录 *之后* 能计算出来的所有内容, 他在看到交互记录 *之前* 也本应能计算出来

交互记录模拟

巧妙的定义

如果Bob无需与Alice交互，也能生成记录，那么这个交互就是零知识的：





只要Bob在看到记录后*之后*能计算出的内容 ...

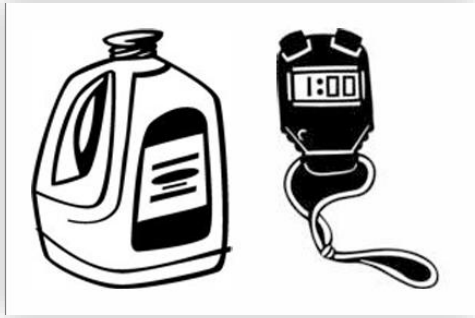
... 就一定存在某种方式，在不交互的情况下也能算出来!

明显的悖论

悖论?

- 交互记录本应让Bob相信某个新事实
- 可是Bob自己也能生成这份记录

Bob

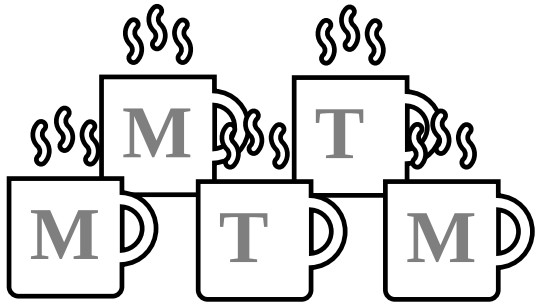


Charlie

B “Alice能在一个小时内喝掉一加仑牛奶!”
C “真的吗？”
B “是的，看这个空牛奶壶和秒表？”
C “笨蛋，谁都能找个空壶和秒表！”
B “但我是亲眼看着她喝的，还亲自计时了！”

明显的悖论

Bob



Charlie

B “Alice能分辨是茶倒进牛奶，还是牛奶倒进茶！”

C “真的吗？”

B “是的，你看这些她全部判断出正确的茶杯？？”

C “笨蛋，谁都能倒杯茶再贴个标签！”

B “但我是随机决定倒茶顺序的，她每次都答对了！”

Bob 原本就知道每个问题的正确答案

- 他之所以相信，是因为交互记录是在回应他的挑战中生成的。

正式定义

定义 [GMR 1985]

零知识证明是满足以下性质的交互式协议:

- **完备性**: 如果陈述为真，证明者总能让验证者信服
- **可靠性**: 如果陈述为假, 即使是证明者作弊，也几乎不能骗过验证者（能骗过的概率极低）
- **零知识性**: 存在一个高效算法，可生成 “看起来一模一样” 的协议交互记录（transcript）

数独零知识证明

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

5	3			7				
			1		5			
		8					6	
8								3
			8					1
7				2				
	6					2		
			4	1				
				8			7	

Alice

Bob, 试一下! 存在一个解,
相信我。

Bob

一个数独零知识证明的工具

刮刮卡 (密码学中的承诺方案)

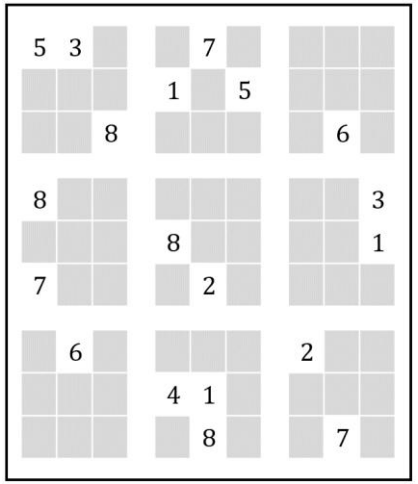
- 隐藏信息
- 信息已经被绑定在卡上
  - 制作完成后不能再修改

假设Alice能制作这样的卡

- 在密码学中, 这可以通过计算 $H(m||r)$实现, m 是要隐藏的信息, r 是随机字符串

数独零知识证明

零知识协议:



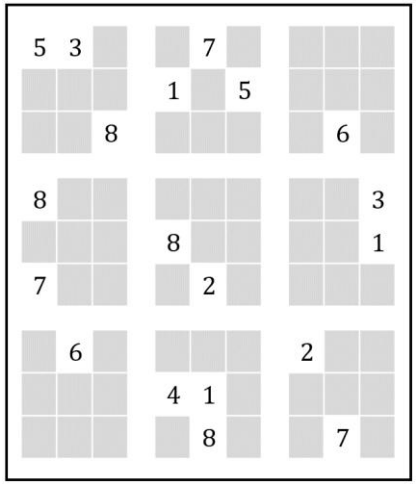
Alice

Bob

数独零知识证明

零知识协议:

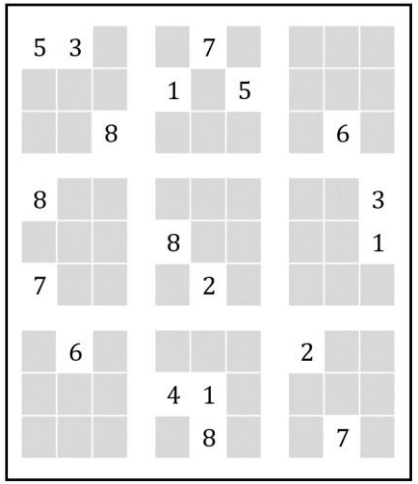
5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9



Alice

Bob

数独零知识证明



零知识协议:

1. Alice 随机重新编号 $\{1, \ldots, 9\}$

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

$1 \longrightarrow 3$
$2 \longrightarrow 8$
$3 \longrightarrow 4$
$4 \longrightarrow 6$
$5 \longrightarrow 9$
$6 \longrightarrow 7$
$7 \longrightarrow 2$
$8 \longrightarrow 1$
$9 \longrightarrow 5$

Alice

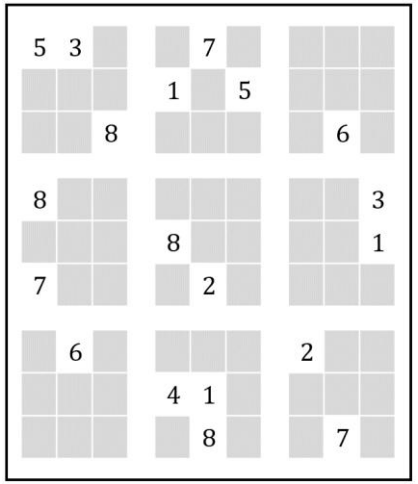
9	4	6	7	2	1	5	3	8
7	2	8	3	5	9	4	6	1
3	5	1	4	6	8	9	7	2
1	9	5	2	7	3	6	8	4
6	8	7	1	9	4	2	5	3
2	3	4	5	8	6	1	9	7
5	7	3	9	4	2	8	1	6
8	1	2	6	3	5	7	4	9
4	6	9	8	1	7	3	2	5

Bob

数独零知识证明

零知识协议:

1. Alice 随机重新编号 $\{1, \ldots, 9\}$



9	4	6	7	2	1	5	3	8
7	2	8	3	5	9	4	6	1
3	5	1	4	6	8	9	7	2
1	9	5	2	7	3	6	8	4
6	8	7	1	9	4	2	5	3
2	3	4	5	8	6	1	9	7
5	7	3	9	4	2	8	1	6
8	1	2	6	3	5	7	4	9
4	6	9	8	1	7	3	2	5

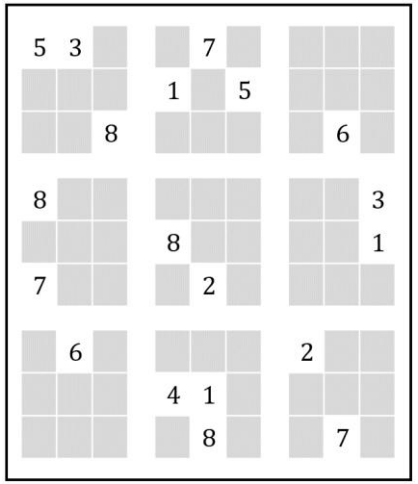
Alice

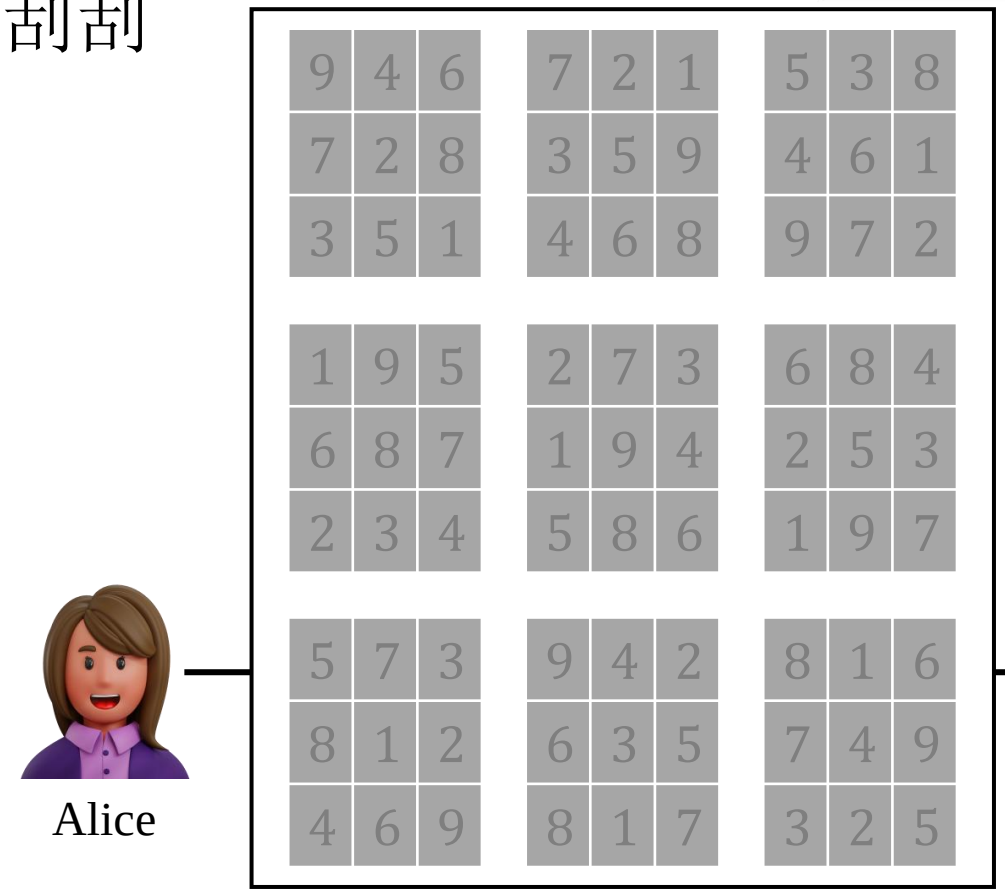
Bob

数独零知识证明

零知识协议:

1. Alice 随机重新编号 $\{1, \ldots, 9\}$
2. Alice 将重新编号后的解填在刮刮卡上, 展示给Bob





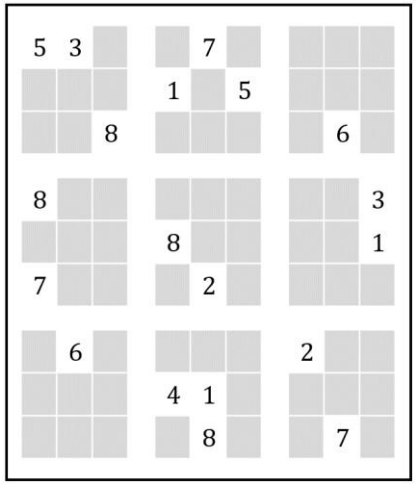
Alice

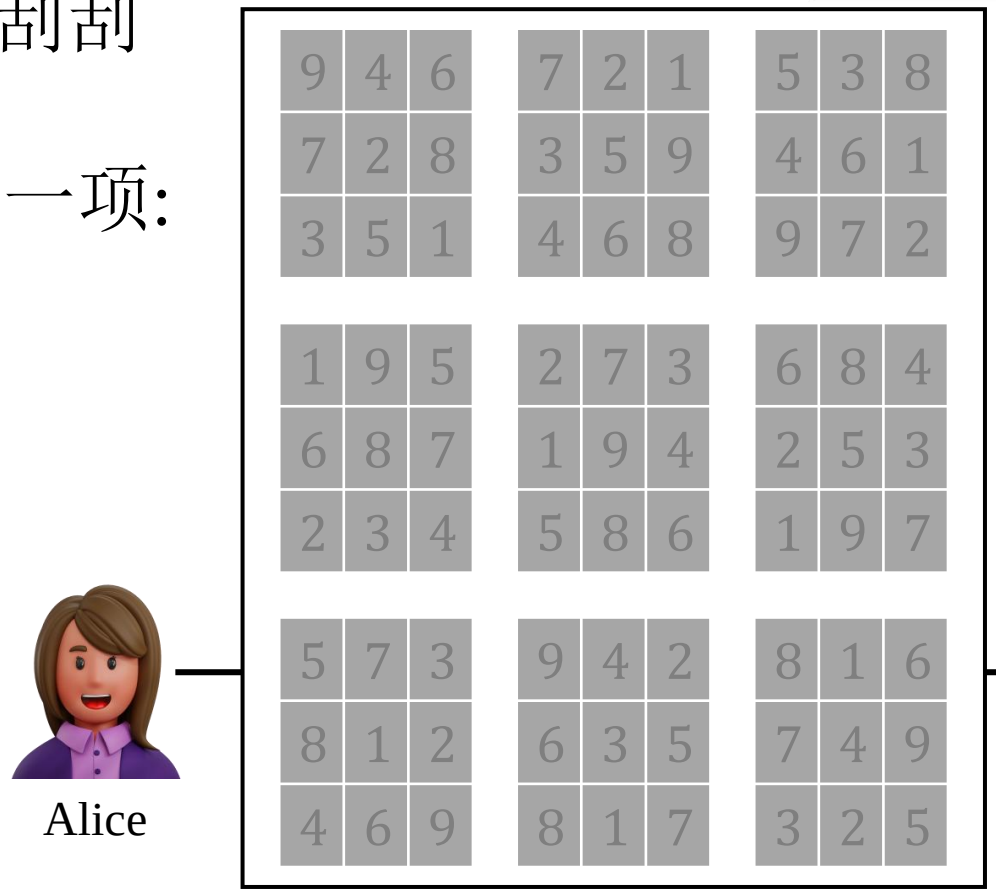
Bob

数独零知识证明

零知识协议:

1. Alice 随机重新编号 $\{1, \ldots, 9\}$
2. Alice 将重新编号后的解填在刮刮卡上, 展示给Bob
3. Bob 要求 Alice 刮开以下任意一项:





Alice

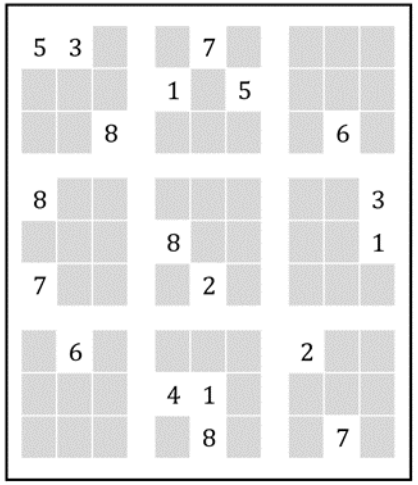
Bob

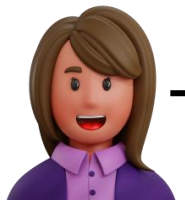
数独零知识证明

零知识协议:

1. Alice 随机重新编号 $\{1, \ldots, 9\}$
2. Alice 将重新编号后的解填在刮刮卡上, 展示给Bob
3. Bob 要求 Alice 刮开以下任意一项:
   - 某一行

并且检查一致性





Alice

9	4	6	7	2	1	5	3	8
7	2	8	3	5	9	4	6	1
3	5	1	4	6	8	9	7	2
1	9	5	2	7	3	6	8	4
6	8	7	1	9	4	2	5	3
2	3	4	5	8	6	1	9	7
5	7	3	9	4	2	8	1	6
8	1	2	6	3	5	7	4	9
4	6	9	8	1	7	3	2	5

Row 2

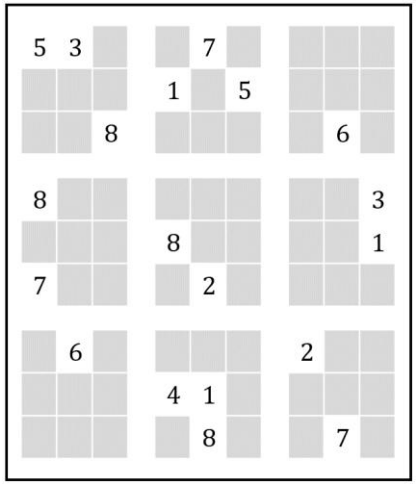
Bob

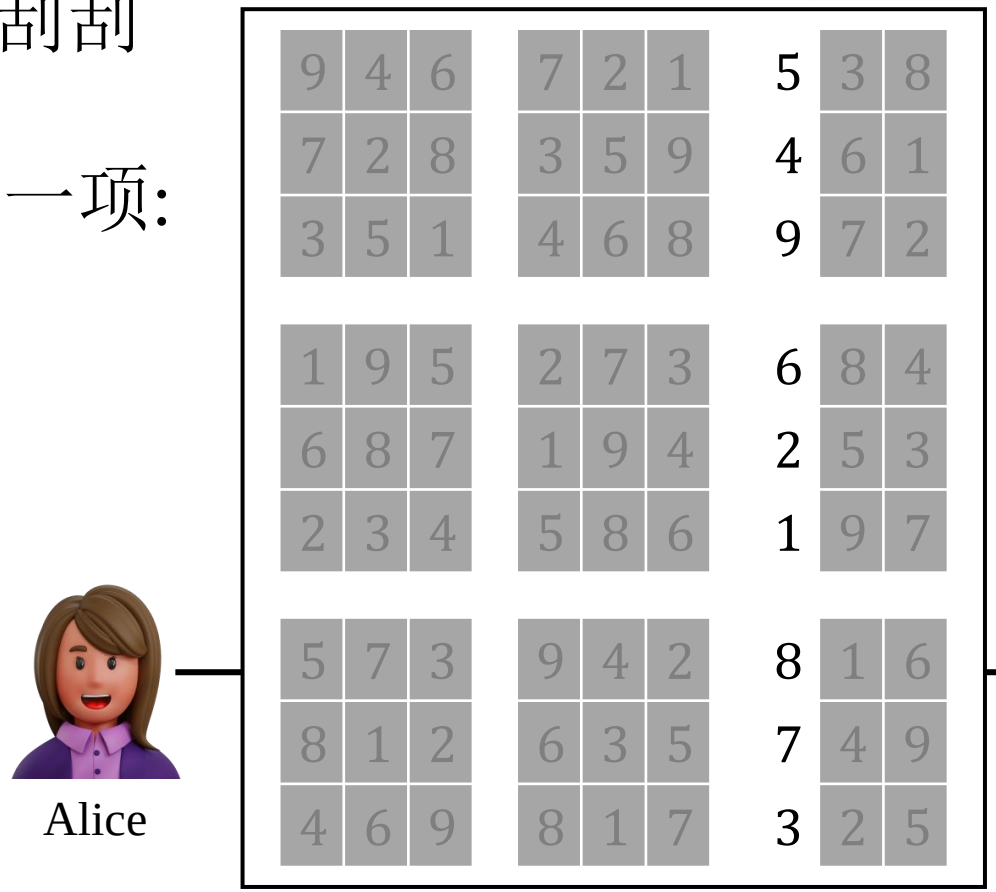
数独零知识证明

零知识协议:

1. Alice 随机重新编号 $\{1, \ldots, 9\}$
2. Alice 将重新编号后的解填在刮刮卡上, 展示给Bob
3. Bob 要求 Alice 刮开以下任意一项:
   - 某一行
   - 某一列

   并且检查一致性





Alice

Col 7

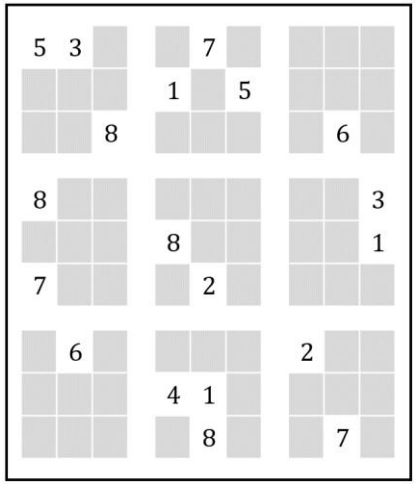
Bob

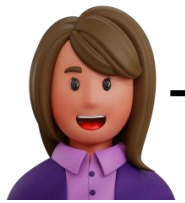
数独零知识证明

零知识协议:

1. Alice 随机重新编号 $\{1, \ldots, 9\}$
2. Alice 将重新编号后的解填在刮刮卡上, 展示给Bob
3. Bob 要求 Alice 刮开以下任意一项:
   - 某一行
   - 某一列
   - 某一个3×3 宫格

   并且检查一致性





Alice

9	4	6	7	2	1	5	3	8
7	2	8	3	5	9	4	6	1
3	5	1	4	6	8	9	7	2
1	9	5	2	7	3	6	8	4
6	8	7	1	9	4	2	5	3
2	3	4	5	8	6	1	9	7
5	7	3	9	4	2	8	1	6
8	1	2	6	3	5	7	4	9
4	6	9	8	1	7	3	2	5

BLK 8

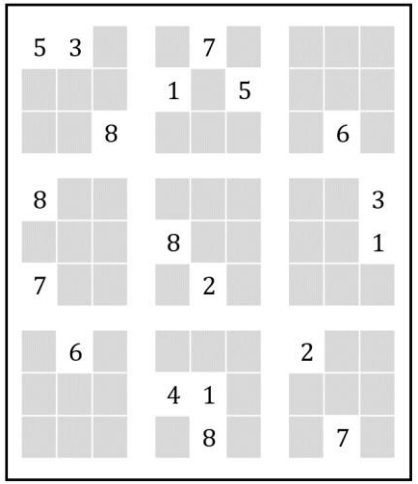
Bob

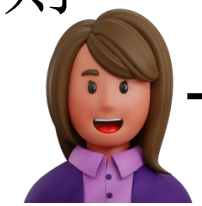
数独零知识证明

零知识协议:

1. Alice 随机重新编号 $\{1, \ldots, 9\}$
2. Alice 将重新编号后的解填在刮刮卡上, 展示给Bob
3. Bob 要求 Alice 刮开以下任意一项:
   - 某一行
   - 某一列
   - 某一个 3×3 宫格
   - 初始位置 + 重新编号的规则

   并且检查一致性
4. 重复执行 n 次





Alice

9	4	6	7	2	1	5	3	8
7	2	8	3	5	9	4	6	1
3	5	1	4	6	8	9	7	2
1	9	5	2	7	3	6	8	4
6	8	7	1	9	4	2	5	3
2	3	4	5	8	6	1	9	7
5	7	3	9	4	2	8	1	6
8	1	2	6	3	5	7	4	9
4	6	9	8	1	7	3	2	5

Bob

数独零知识证明, 分析

观察 (完备性)

如果Alice能成功应对所有挑战, 那么她的刮刮卡满足:

- 每行, 每列, 每宫 均为 $\{1, \ldots, 9\}$的排列
- 初始位置与原数独的 $\{1, \ldots, 9\}$ 重新编号的一致

那么原始数独确实有解.

如果Alice作弊 (实际无解)?
$\Rightarrow$ 不存在能通过所有挑战的刮刮卡。

数独零知识证明, 分析

假设 Alice 试图证明一个错误的陈述。 设 c 是对 Alice 的刮刮卡不利的一个挑战。

Bob 从 28 个挑战中随机选择一个

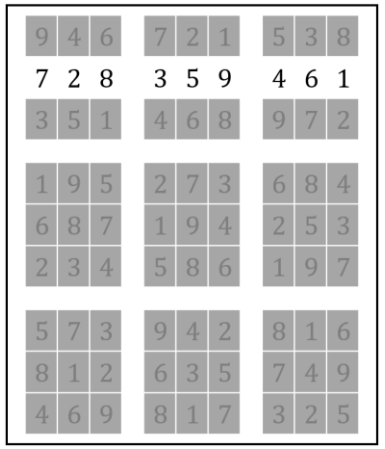
- $1/28$的概率, Bob 选中 c， Alice被识破！
- 以不超过 $\leq 27/28$的概率, Alice的欺骗未被发现。

核心思路

重复协议 n 次. Alice的欺骗每轮都未被发现的概率为 $(27/28)^n \approx (1/2)^{0.05n}$

当$n = 2500$, Alice被抓到的概率约为99% (可靠性)

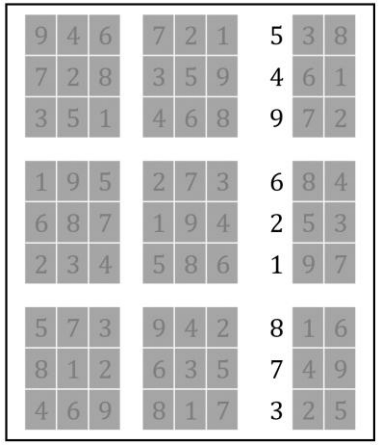
数独零知识证明, 分析



如果 Alice 遵循协议（即确实有解），那么每一轮的交互记录是：

- 一个随机行中 $\{1, \ldots, 9\}$ 的随机排列,

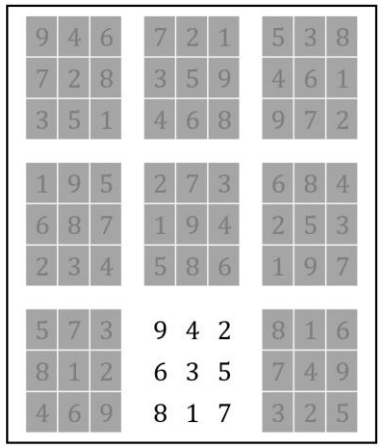
数独零知识证明, 分析



如果 Alice 遵循协议（即确实有解），那么每一轮的交互记录是：

- 一个随机行中 $\{1, \ldots, 9\}$ 的随机排列,
- 一个随机列中 $\{1, \ldots, 9\}$ 的随机排列,

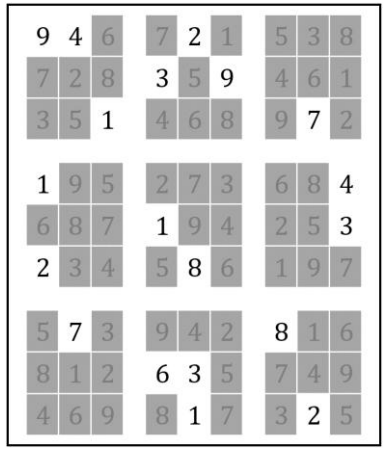
数独零知识证明, 分析



如果 Alice 遵循协议（即确实有解），那么每一轮的交互记录是：

- 一个随机行中 $\{1, \ldots, 9\}$ 的随机排列,
- 一个随机列中 $\{1, \ldots, 9\}$ 的随机排列,
- 一个随机九宫格中 $\{1, \ldots, 9\}$ 的随机排列, 或

数独零知识证明, 分析



如果 Alice 遵循协议（即确实有解），那么每一轮的交互记录是：

- 一个随机行中 $\{1, \ldots, 9\}$ 的随机排列,
- 一个随机列中 $\{1, \ldots, 9\}$ 的随机排列,
- 一个随机九宫格中 $\{1, \ldots, 9\}$ 的随机排列, 或
- 对原始题目的初始位置的随机编号。

这些内容Bob自己也能生成 (不知道解)! (零知识)

万物皆可零知识证明？

我们已经有了用于数独的零知识证明协议，那又怎样？

定理 [Yato 2003] $n \times n$ 的数独是NP完全的。

每一个 （实际的）陈述都可以转化为一个（广义）数独实例的可解性问题

- 给定陈述 x, 可以计算出一个数独谜题 $S(x)$
- x 为真 $\Leftrightarrow S(x)$ 是一个可解的数独
- 要证明 x, 就对 $S(x)$进行数独的零知识证明

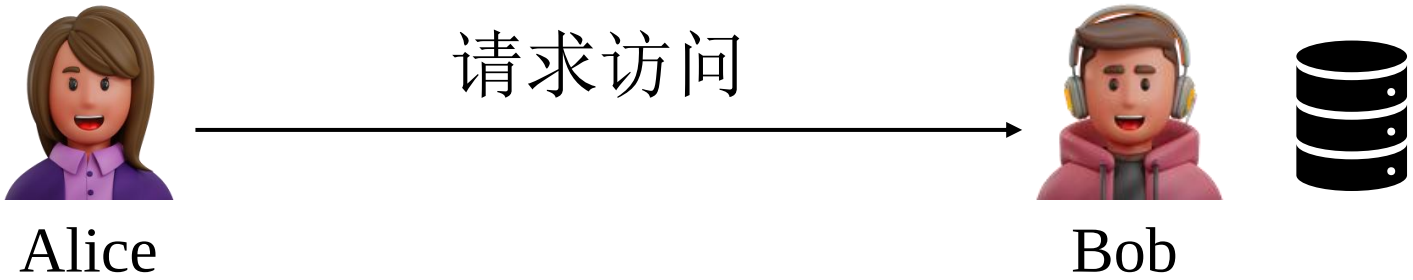
定理 每一个NP陈述都可以用零知识方式进行证明

这些有什么用呢?

零知识

公钥数据库:

Alice: 583ffb4b3..
Bob: 0fae6535d..
Charlie: 0dd1dd8de..



零知识身份验证

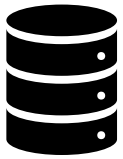
公钥数据库:

Alice: 583ffb4b3..
Bob: 0fae6535d..
Charlie: 0dd1dd8de..

Alice

零知识证明: 我知道对应于 583ffb4b3..的私钥

Bob



零知识身份验证

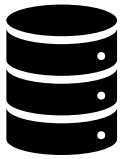
公钥数据库:

Alice:	583ffb4b3..
Bob:	0fae6535d..
Charlie:	0dd1dd8de..

Alice

好的, Alice

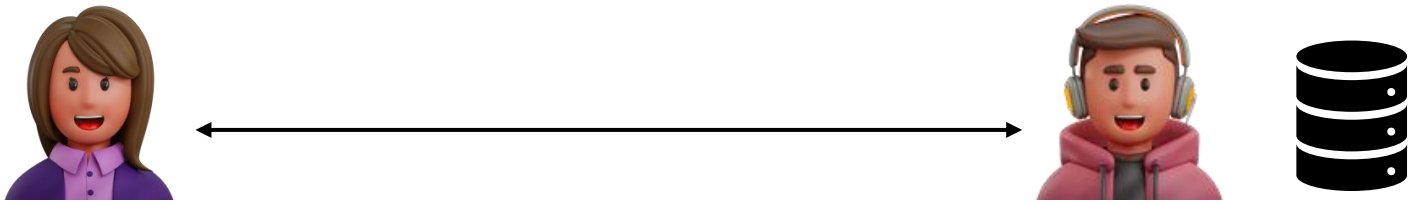
Bob



零知识身份验证

公钥数据库:

Alice: 583ffb4b3..
Bob: 0fae6535d..
Charlie: 0dd1dd8de..



- Alice的公钥已公开
- 没有人知道（也无法计算出）与之对应的私钥

安全性 “编译器”

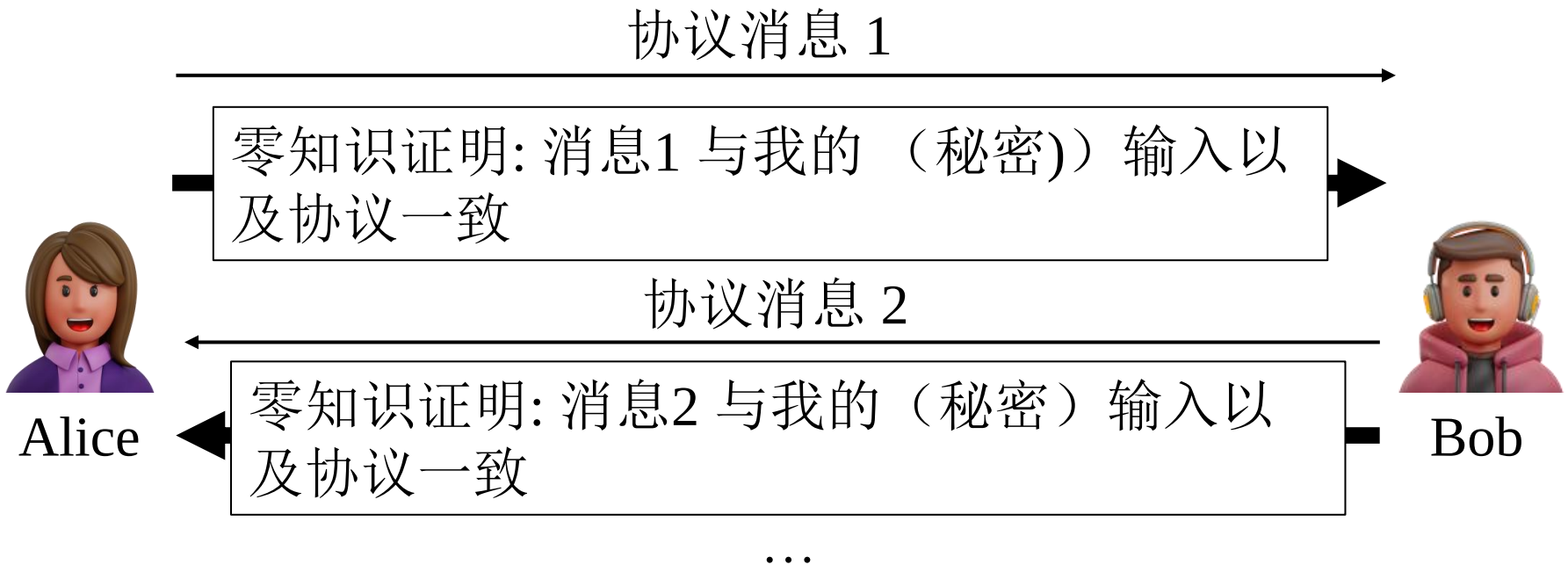
问题

- 希望协议能提供安全性保障, 即使是面对恶意参与方
- 这很难!
- 假设所有参与方都遵守协议会更简单

巧妙想法: 安全性编译器 [GMW 1987]

1. 设计一个各方都遵守协议时安全的协议 (相对容易)
2. 参与方必须在每一步都证明自己确实遵守了协议

安全性 “编译器”



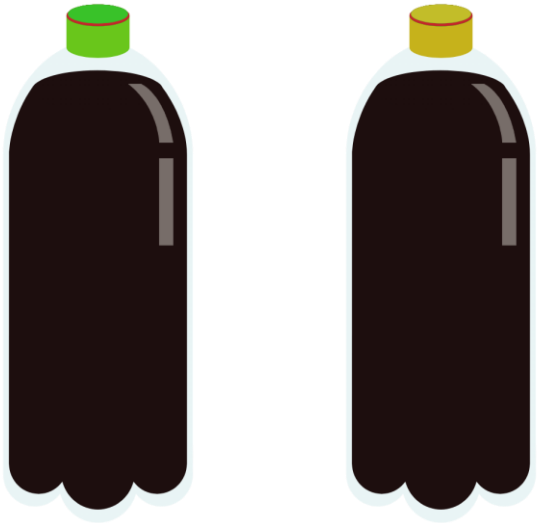
- 零知识证明不会泄露关于秘密输入的其他信息
- 如果证明通过，说明各方诚实地执行了协议
  - 安全性得以保证
- 如果证明失败，终止协议！

其他应用

- 区块链
  - 隐藏发送方
  - 隐藏接收方
  - 隐藏交易金额
- 投票
- ……

思考一下…

- 你能说出百事可乐和可口可乐的区别吗？
- 给出两瓶可乐 (一瓶可口可乐; 一瓶百事可乐)，证明你能区分它们。
- 如何设计零知识证明协议？



安全多方计算
Secure Multi-Party Computation

安全多方计算

x_1

x_2

x_5

x_3

x_4

前提:

- 互不信任的参与方, 各自拥有私密输入

安全多方计算

x_1

x_2

$f(x_1, x_2, x_3, x_4, x_5)$

x_5

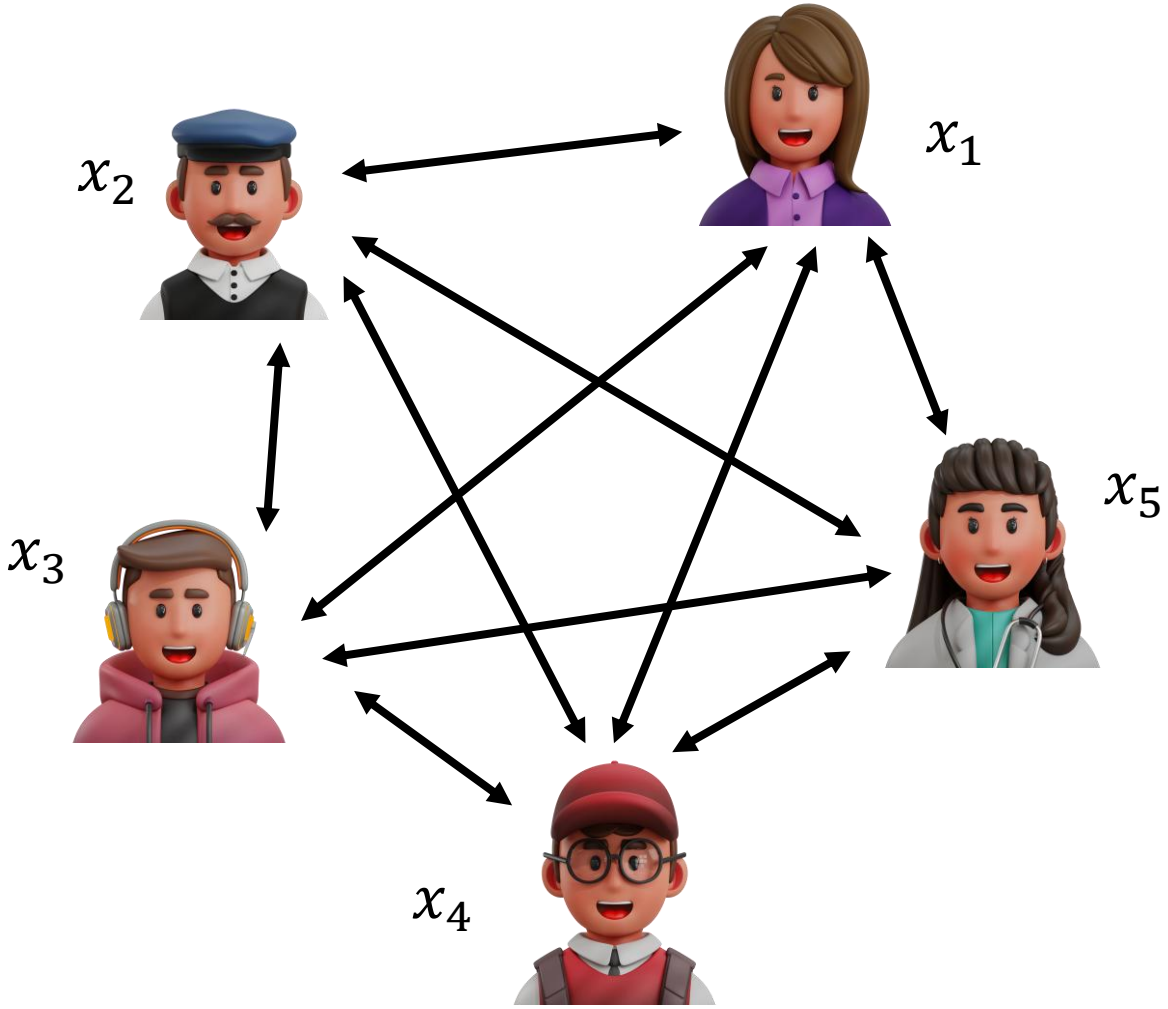
x_3

x_4

前提:

- 互不信任的参与方, 各自拥有私密输入
- 希望共同计算一个约定的结果
- 例如: 选举, 拍卖等

安全多方计算



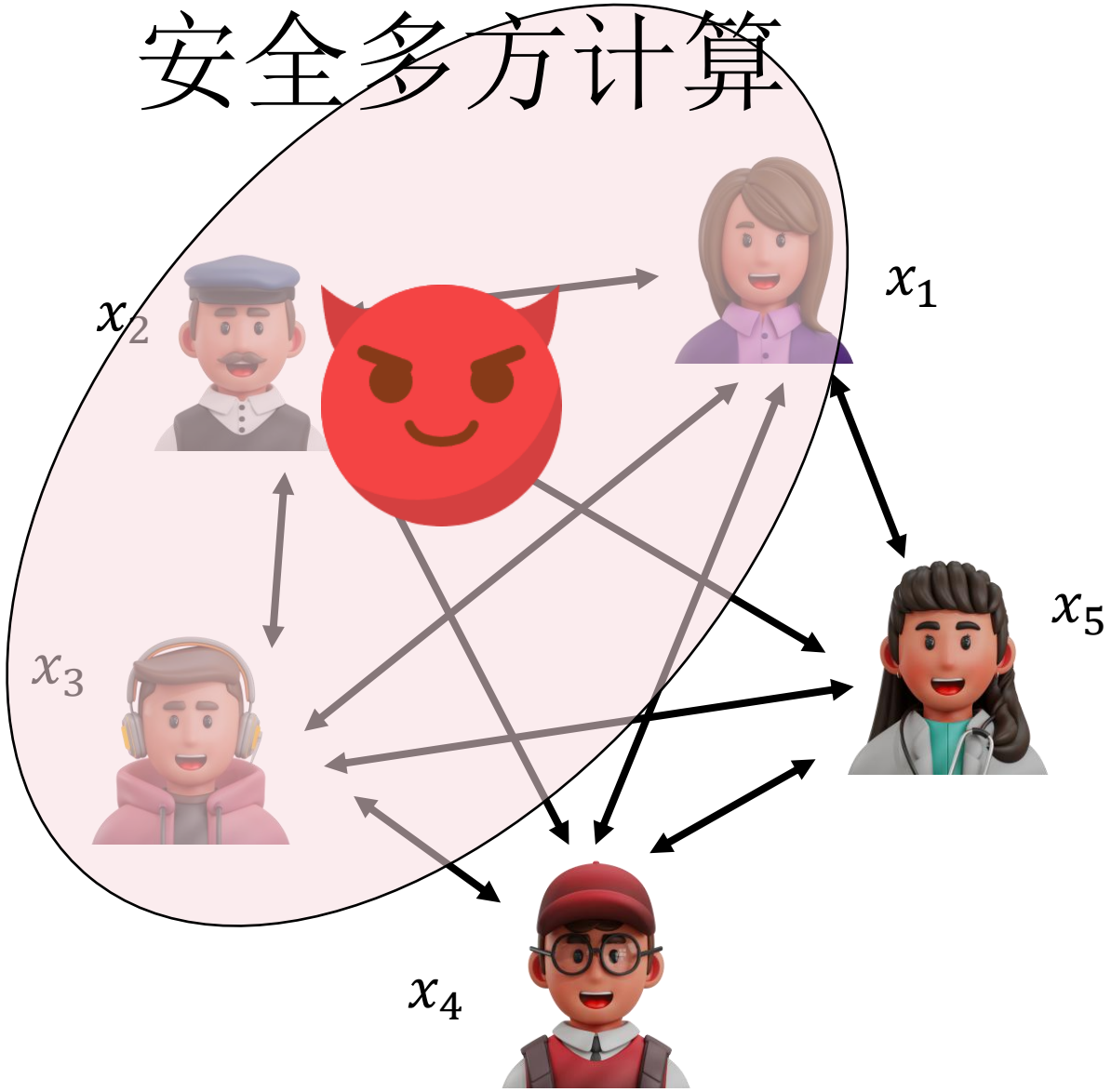
前提:

- 互不信任的参与方, 各自拥有私密输入
- 希望共同计算一个约定的结果
- 例如: 选举, 拍卖等

安全性保障:

- 隐私性 (只能获得协议规定的输出)
- 输入独立性
- 输出一致性，等等

安全多方计算



前提:

- 互不信任的参与方, 各自拥有私密输入
- 希望共同计算一个约定的结果
- 例如: 选举, 拍卖等

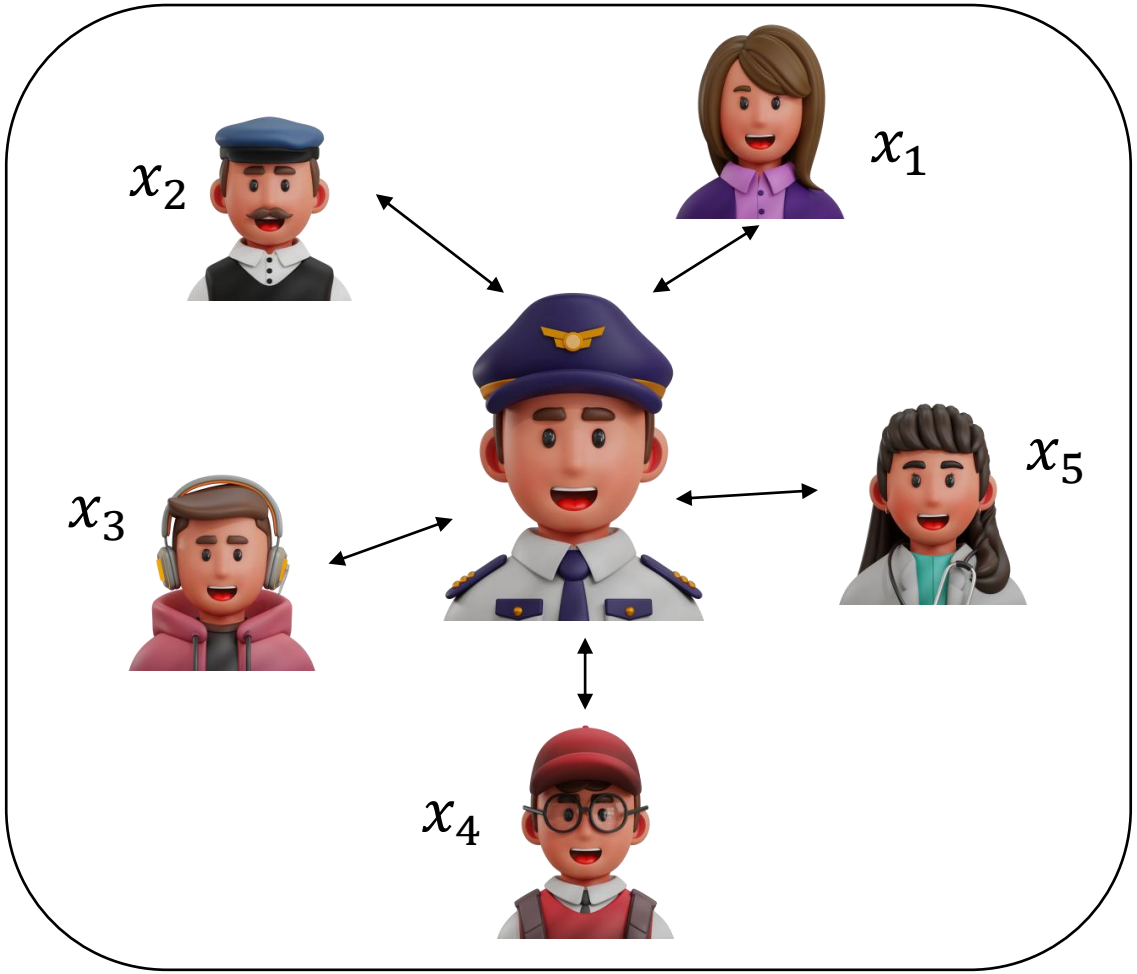
安全性保障:

- 隐私性 (只能获得协议规定的输出)
- 输入独立性
- 输出一致性，等等

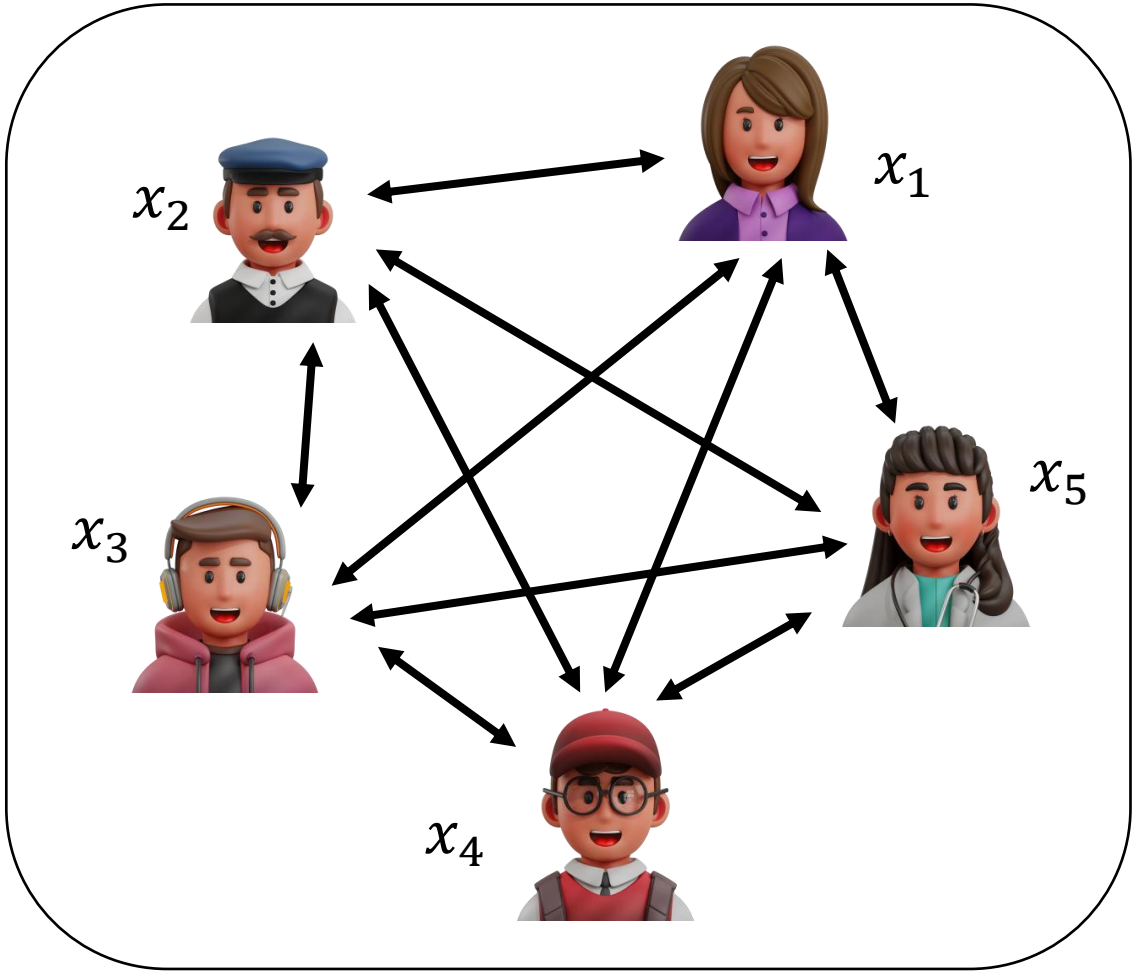
..即使有参与方作弊, 串通!

安全多方计算的目标

- 设计一个协议来模拟可信任第三方的存在

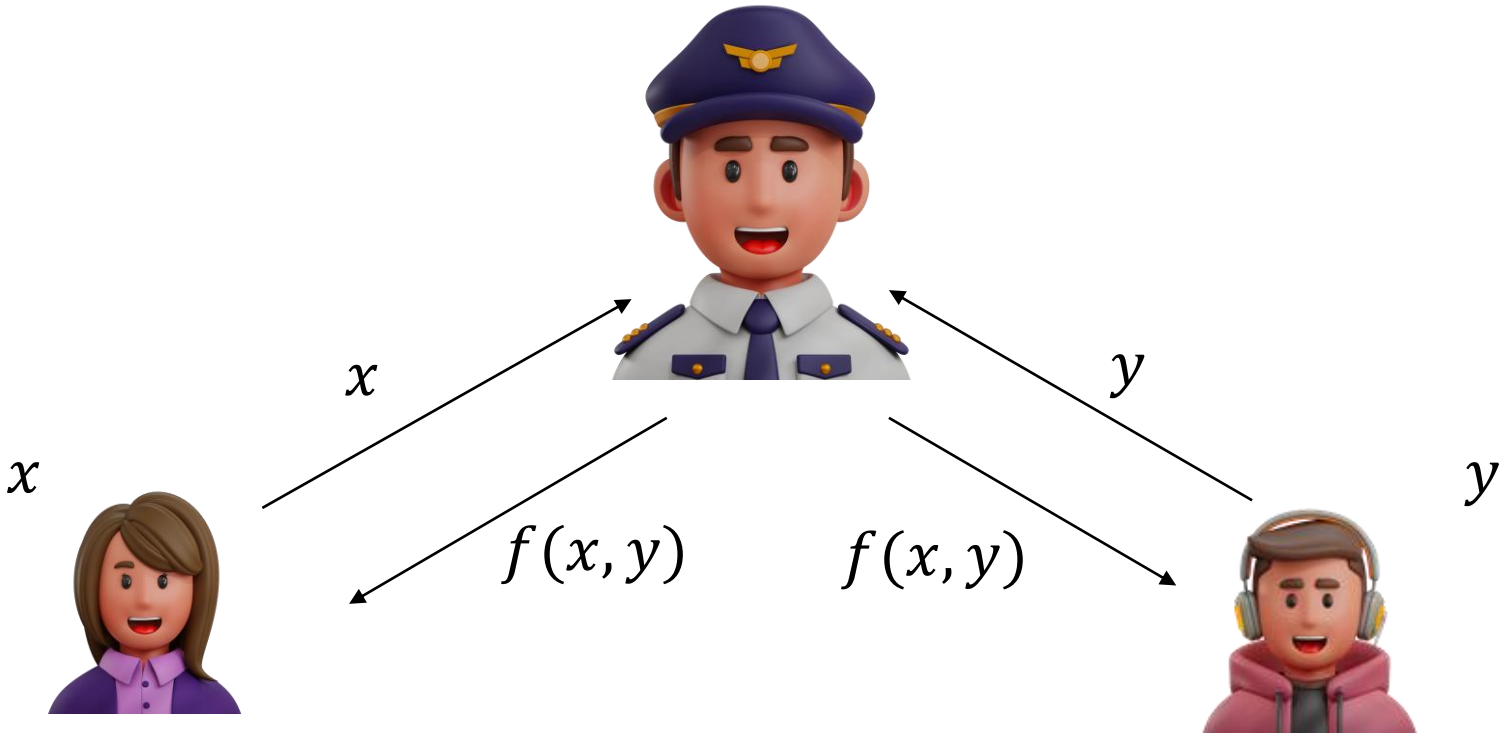


$\equiv$



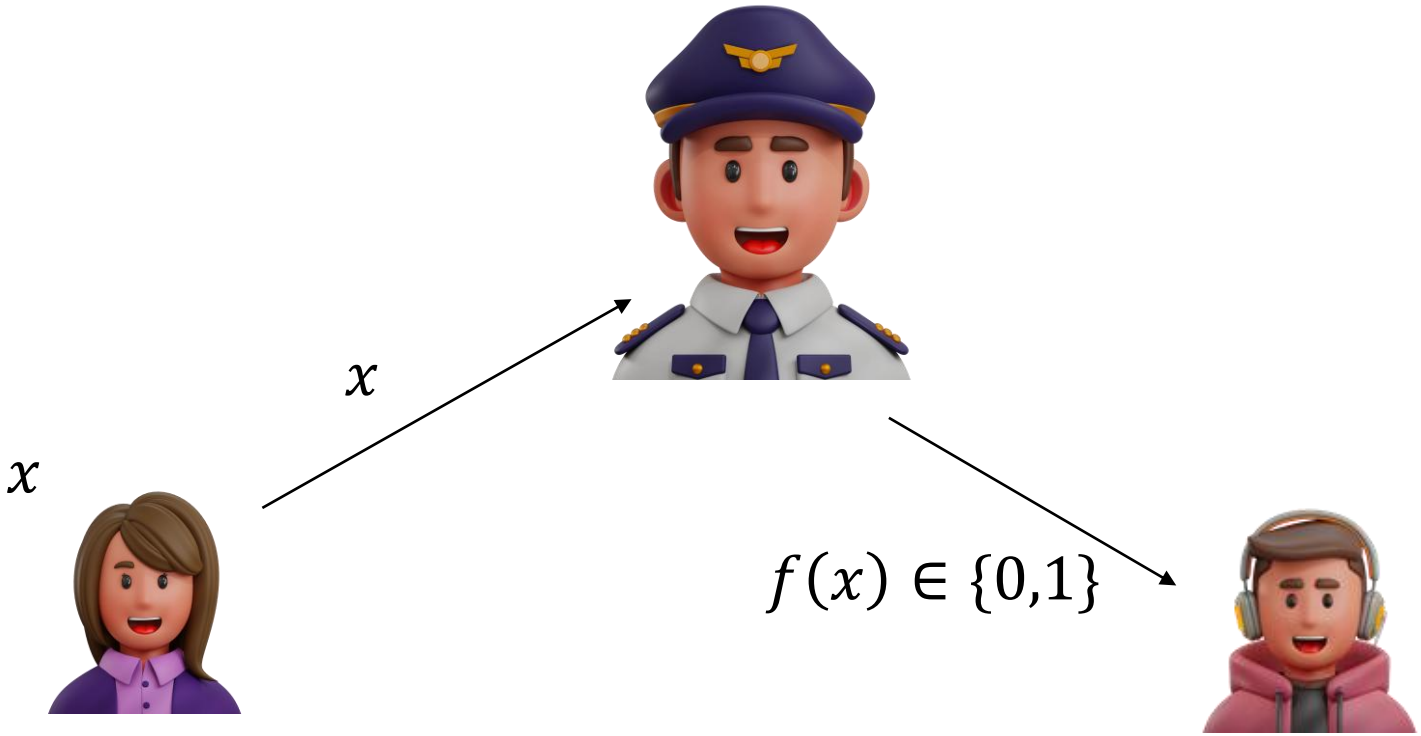
零知识证明——特例

- 参与方数量: 2
- 其中一个参与方没有输入!
- 输出长度是 1 bit!



零知识证明——特例

- 参与方数量: 2
- 其中一个参与方没有输入!
- 输出长度是 1 bit!



安全多方计算

Protocols for Secure Computations
(extended abstract)

Andrew C. Yao
University of California Berkeley, California 94720

1 Introduction

Two millionaires wish to know who is richer; however, they do not want to find out inadvertently any additional information about each other's wealth. How can they carry out such a conversation?

This is a special case of the following general problem. Suppose m people wish to compute the value of a function $f(x_1, x_2, x_3, \ldots, x_m)$, which is an integer-valued function of m integer variables x_i of bounded range. Assume initially person P_i knows the value of x_i and no other x's. Is it possible for them to compute the value of f, by communicating among themselves, without unduly giving away any information about the values of their own variables? The millionaires' problem corresponds to the case when $m = 2$ and $f(x_1, x_2) = 1$ if $x_1 < x_2$, and 0 otherwise. In this paper, we will give precise formulation of this general problem and describe three ways of solving it by use of one-way functions (i.e., functions which are easy to evaluate but hard to invert). These results have applications to secret voting, private querying of database, oblivious negotiation, playing mental poker, etc. We will also discuss the complexity question "How many bits need to be exchanged for the computation", and describe methods to prevent participants from cheating. Finally, we study the question "What cannot be accomplished with one-way functions".

Before describing these results, we would like to put this work in perspective by first considering a unified view of secure computation in the next section.

2 A Unified View of Secure Computation

Since one-way functions were first proposed in 1976 (Diffie and Hellman [1]), they have been used in two kinds of applications. The first kind is concerned with the encryption and transmission of messages to make them unreadable and unalterable for eavesdroppers and saboteurs [1, 2, 3, 4]. The second kind of applications includes "mental poker" (Shamir, et.al. [5]), in which two players deal cards by communicating over a telephone line, and "coin flipping" (Blum [6]), in which two mutually suspecting parties are to generate an unbiased bit. It would be desirable to have a unified framework where all these applications can be related, and where common proof techniques can be developed for proving the security of protocols. More fundamentally, such a framework is essential if we are ever to understand the intrinsic power and limitation of one-way functions. For example, without a precise model it would be hard to answer a question such as "Is it possible for three mutually suspecting parties to interactively generate a bit with bias $1/\varepsilon$?"

In response to this need, we propose to adopt the following view. Two parties Alice and Bob, in possession of private variables i and j respectively, wish to communicate so that Alice can evaluate a function $f(i, j)$, and Bob a function $g(i, j)$. There may be some eavesdroppers or saboteurs on the communication line. The purpose of a protocol would be to design an algorithm for Alice and Bob to follow, such that certain *security constraints* (against saboteur) and *privacy constraints* (Alice may not wish to reveal the exact value of i) can be satisfied.

In one extreme, when the computation component is trivial, e.g. if f = constant and $g(i, j) = i$, then we get the first kind of applications mentioned before, in which the basic concern is eavesdropping and sabotage. In the other extreme, when such external threats can be ignored, but the computation of f and g is nontrivial, then we get the problem which is to be studied in this paper. (Mental poker and coin flipping represent a stochastic version of this problem which will also be discussed.) Note that, although we have used Alice and Bob in the above description, all discussions can be extended to the case of m parties communicating.

It would be natural to discuss these two special cases together. However, due to length considertion, we will report here only the results corresponding to the computation-intense case with no outside saboteurs. Results on the other case will be reported elsewhere.

3 Deterministic Computations

3.1 Solutions to the Millionaires' Problem

In this abstract, we will describe in detail only one of the three solutions we have.

For definiteness, suppose Alice has i millions and Bob has j millions, where $1 < i, j < 10$. We need a protocol for them to decide whether $i < j$, such that this is also the only thing they know in the end (aside from their own values). Let M be the set of all N-bit nonnegative integers, and Q_N be the set of all 1-1 onto functions from M to M. Let E_n be the public key of Alice, generated by choosing a random element from Q_N.

The protocol proceeds as follows:

1

0272-5428/82/0000/0160$00.75 © 1982 IEEE

Protocols for secure computations, FOCS 82

How to Generate and Exchange Secrets
(extended abstract)

Andrew Chi-Chih Yao*
Department of Computer Science
Princeton University
Princeton, New Jersey 08544

Abstract

In this paper we introduce a new tool for controlling the knowledge transfer process in cryptographic protocol design. It is applied to solve a general class of problems which include most of the two-party cryptographic problems in the literature.

Specifically, we show how two parties A and B can interactively generate a random integer $N = p \cdot q$ such that its *secret*, i.e., the prime factors (p, q), is hidden from either party individually but is recoverable jointly if desired. This can be utilized to give a protocol for two parties with private values i and j to compute any polynomially computable functions $f(i, j)$ and $g(i, j)$ with minimal knowledge transfer and a strong *fairness* property. As a special case, A and B can exchange a pair of secrets s_A, s_B, e.g. the factorization of an integer and a Hamiltonian circuit in a graph, in such a way that s_A becomes computable by B when and only when s_B becomes computable by A. All these results are proved assuming only that the problem of factoring large intergers is computationally intractable.

1. Introduction.

A *protocol* $\mathcal{M} = (M_A, M_B)$ is a pair of communicating probabilistic Turing machines each with a special "send-receive"-tape. Given inputs of the form (n, i_A) and (n, i_B), the two machines will alternately send and receive message strings using the send-receive tapes; each machine will perform computation as a standard Turing machine after receiving a message string, including the computation of the next message to be sent. Eventually both machines halt within a number of steps bounded by some polynomial in n, leaving strings u_A and u_B on the output tapes. For a detailed description, see for example [GMR] [GHY]; in our case we allow each machine to have their own private input tape. For any run σ of the protocol, let $\Delta_A(\sigma)$ denote the *history* of the run from A's view, i.e. a sequence of the instataneous descriptions of M_A; similarly $\Delta_B(\sigma)$ denote the history of this run from B's view.

The class of problems we are interested in as follows. Given inputs (n, i_A) and (n, i_B), where (i_A, i_B) is a random pair of strings distributed according to a probability distribution h_n over $\{0,1\}^* \times \{0,1\}^*$, we wish to design a protocol $\mathcal{M} = (M_A, M_B)$ such that the outputs (u_A, u_B) is distributed according to a certain probability distribution w_{n,i_A,i_B} over $\{0,1\}^* \times \{0,1\}^*$. The sequence of distributions $(h_1, h_2, \ldots, h_n, \ldots)$ is assumed to be a *polynomial ensemble* in the sense that, there is a probabilistic Turing machine which, given input n, will generate in time polynomial in n a random string x with a distribution indistinguishable computationally from h_n; similarly we assume that w_{n,i_A,i_B} is a polynomial ensemble in that a random sample point can be generated in time polynomial in n, when the parameters n, i_A, i_B have been given. Let us call this a *(two-party) interactive computational problem* $\langle h_n, w_n \rangle$. Of special interest is the case when the probability distribution w_{n,i_A,i_B} is nonzero only at one point, call it $(f_n(i_A, i_B), g_n(i_A, i_B))$, in which case we can regard the problem as the evaluation of a pair of functions $f_n(i_A, i_B), g_n(i_A, i_B)$; write it as $\langle h_n, (f_n, g_n) \rangle$.

In addition to the *validity* requirement above, we would like the protocol to have certain *privacy* and *fairness* properties. Roughly, "privacy" means that if A behaves according to the protocol, then B will have no more information about the values of i_A, u_A than in the situation where an oracle does the computation for B and just hands B a value u_B. Classical examples where privacy is the main concern include oblivious transfer (Rabin [R], Fischer et al [FRMW]), coin tossing (Blum [B1], Cleve [C]), mental poker (Shamir, Rivest, and Adleman [SRA], Goldwasser and Micali [GM]). A general study of this problem is given in Yao [Y1,Y2]. A related problem is the *interactive proof system* (Goldwasser, Micali, and Rackoff [GMR], Galil, Haber, and Yung [GHY]) in which player A wishes to convince player B that a string r is in a certain language L. Recently, Goldreich, Micali, and Wigderson [GMW] proved that, for players with polynomial-time computing power, any language in NP has a *minimum knowledge interactive proof*, assuming the existence of suitable one-way functions. This latter problem is closely related to, in our formulation, the case of computing a pair of functions $f_n(i_A, i_B), g_n(i_A, i_B)$, where $i_A = (s, r)$, $i_B = r$, $g_n(i_A, i_B) = 1$ if and only if s is a short *proof* for $r \in L$, and $f_n(i_A, i_B) \equiv 1$.

* This research was supported in part by National Science Foundation under grants DCR-83-08109.

0272-5428/86/0000/0162$01.00 © 1986 IEEE

162

Authorized licensed use limited to: MIT Libraries. Downloaded on November 20,2023 at 18:50:39 UTC from IEEE Xplore. Restrictions apply.

How to Generate and Exchange Secrets, FOCS 86

Andrew Chi-Chih Yao
Turing Award (2000)

姚氏百万富翁问题

$\$x$

$\$y$

是否$x > y$?

示例：广告转化率

- 特斯拉在 Google 上投放广告，两家公司想知道广告转化率

Ad impressions
alice@gmail.com
bob@gmail.com
charlie@gmail.com
frank@gmail.com
gina@gmail.com
……

In-store purchases	
albert@gmail.com	$80k
bob@gmail.com	$160k
caroline@gmail.com	$99k
frank@gmail.com	$85k
felipe@gmail.com	$77k
……	

```
SELECT  SUM(amount)
FROM    ads, purchases
WHERE   ads.email = purchases.email
```

示例：广告转化率

- 特斯拉在 Google 上投放广告，两家公司想知道广告转化率

Ad impressions
alice@gmail.com
bob@gmail.com
charlie@gmail.com
frank@gmail.com
gina@gmail.com
……

In-store purchases	
albert@gmail.com	$80k
bob@gmail.com	**$160k**
caroline@gmail.com	$99k
frank@gmail.com	**$85k**
felipe@gmail.com	$77k
……	

```
SELECT  SUM(amount)
FROM    ads, purchases
WHERE   ads.email = purchases.email
```

通过 Google 和客户之间的安全计算来完成

示例：工资公平研究

The New York Times

How Boston Is Trying to Close the Gender Pay Gap

Through pay-negotiation workshops and partnerships with more than 100 companies, the city is trying to help female workers match the salaries of male counterparts.

BOSTON WOMEN'S WORKFORCE COUNCIL REPORT 2017

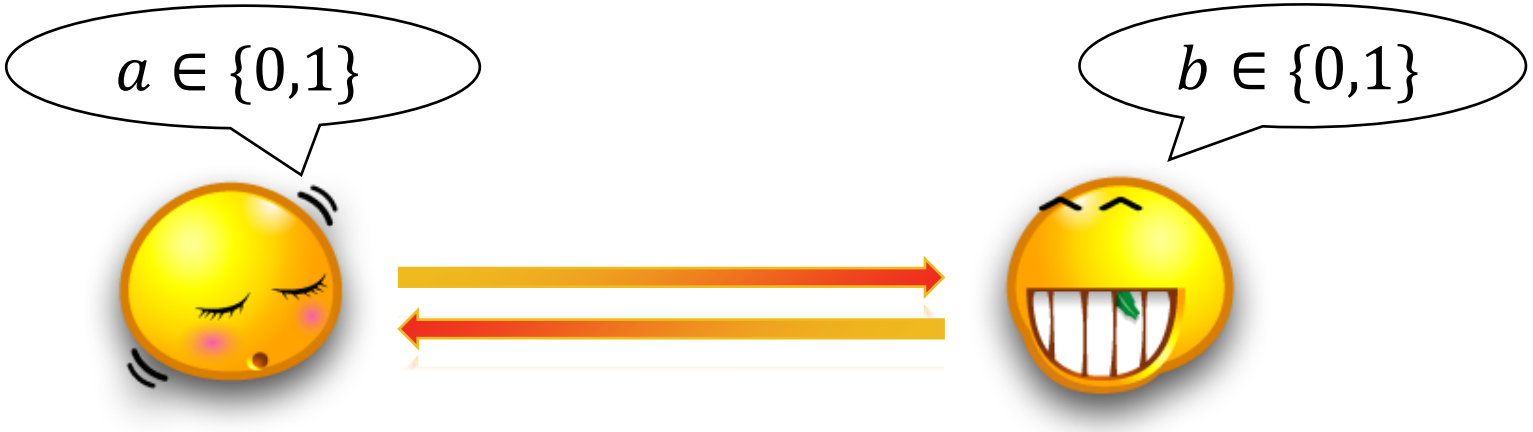
DATA SUBMISSION PROCESS:

Part of the commitment employers make when signing the Boston 100% Talent Compact is to anonymously report employee data to the BWWC biennially. The Software & Application Innovation Lab at Boston University's Rafik B. Hariri Institute of Computing and Computational Science & Engineering, the BWWC's data partner, developed a completely confidential reporting system from which anonymous data from multiple independent sources can be analyzed in the aggregate.

During the submission process, Compact signers submit their wage data in the aggregate form over a unique, web-based software program that employs encryption using a technique known as secure multi-party computation. During this process, individual compensation data never leaves each organization's server. The BWWC then receives aggregate data unconnected to any firm.

示例：约会

- Alice 和 Bob
- 如果他们彼此喜欢, 就可以约会; 否则不约会。
- 若一方不喜欢对方， 则该方不应得知另一方的真实态度

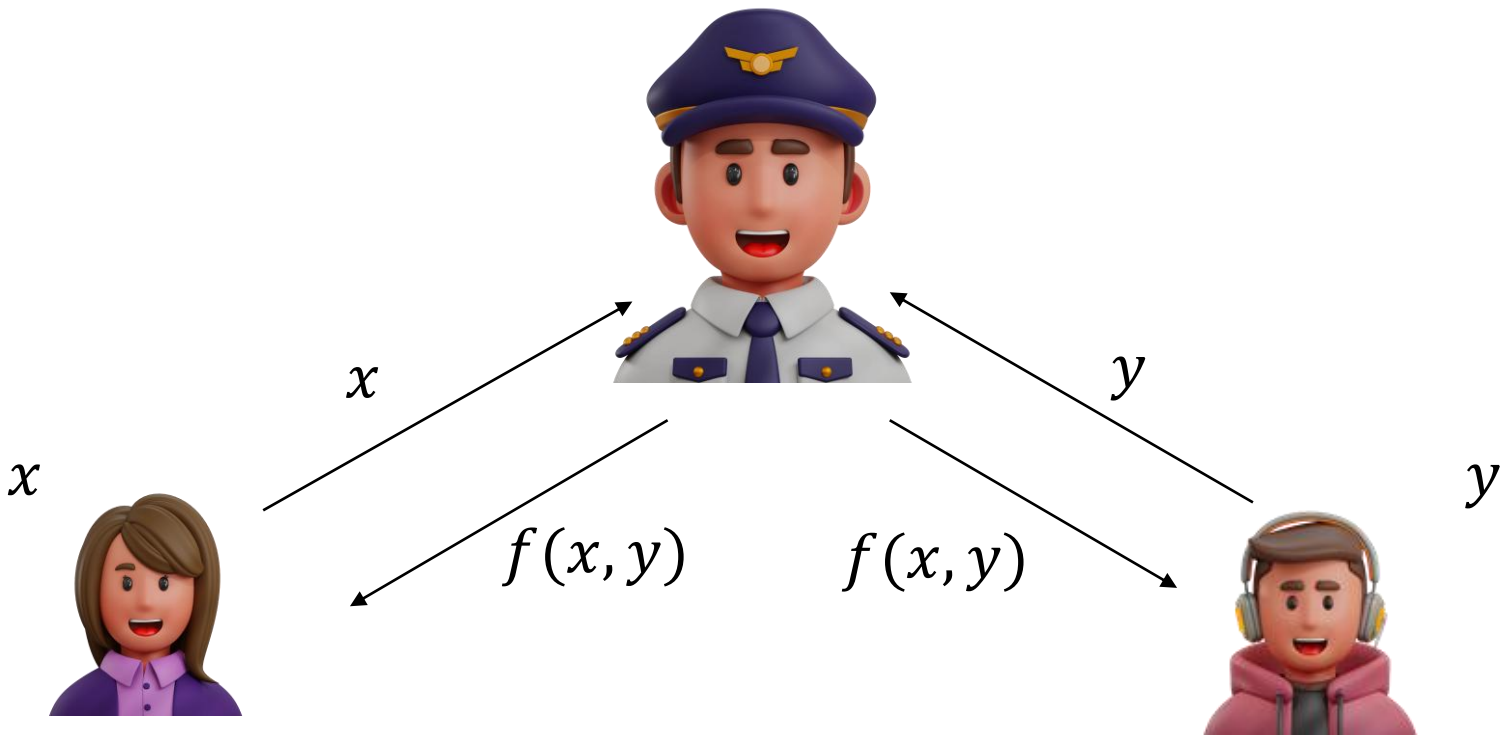


Alice	Bob	Can date?
0	0	0
0	1	0
1	0	0
1	1	1

通用两方安全计算

混淆电路 (Garbled Circuits) + 不经意传输 (Oblivious Transfer)

两个参与方的协议



热身: 混淆真值表 (garbled truth table)

- Alice的输入: $x \in \{1, \ldots, 4\}$
- Bob的输入: $y \in \{1, \ldots, 4\}$
- 他们希望安全地计算函数 $f(x, y)$

热身: 混淆真值表

Alice的操作如下:

1. 写出函数 f 的真值表

1	1	$f(1,1)$
1	2	$f(1,2)$
1	3	$f(1,3)$
1	4	$f(1,4)$
2	1	$f(2,1)$
2	2	$f(2,2)$
2	3	$f(2,3)$
2	4	$f(2,4)$
3	1	$f(3,1)$
3	2	$f(3,2)$
3	3	$f(3,3)$
3	4	$f(3,4)$
4	1	$f(4,1)$
4	2	$f(4,2)$
4	3	$f(4,3)$
4	4	$f(4,4)$

热身: 混淆真值表

Alice的操作如下:

1. 写出函数 f 的真值表
2. 对每个可能的输入, 选择一个随机的**加密密钥**

A_1	B_1	$f(1,1)$
A_1	B_2	$f(1,2)$
A_1	B_3	$f(1,3)$
A_1	B_4	$f(1,4)$
A_2	B_1	$f(2,1)$
A_2	B_2	$f(2,2)$
A_2	B_3	$f(2,3)$
A_2	B_4	$f(2,4)$
A_3	B_1	$f(3,1)$
A_3	B_2	$f(3,2)$
A_3	B_3	$f(3,3)$
A_3	B_4	$f(3,4)$
A_4	B_1	$f(4,1)$
A_4	B_2	$f(4,2)$
A_4	B_3	$f(4,3)$
A_4	B_4	$f(4,4)$

热身: 混淆真值表

Alice的操作如下:

1. 写出函数 f 的真值表
2. 对每个可能的输入, 选择一个随机的**加密密钥**
3. 使用对应的密钥对输出进行加密

$\mathbb{E}_{A_1,B_1}(f(1,1))$
$\mathbb{E}_{A_1,B_2}(f(1,2))$
$\mathbb{E}_{A_1,B_3}(f(1,3))$
$\mathbb{E}_{A_1,B_4}(f(1,4))$
$\mathbb{E}_{A_2,B_1}(f(2,1))$
$\mathbb{E}_{A_2,B_2}(f(2,2))$
$\mathbb{E}_{A_2,B_3}(f(2,3))$
$\mathbb{E}_{A_2,B_4}(f(2,4))$
$\mathbb{E}_{A_3,B_1}(f(3,1))$
$\mathbb{E}_{A_3,B_2}(f(3,2))$
$\mathbb{E}_{A_3,B_3}(f(3,3))$
$\mathbb{E}_{A_3,B_4}(f(3,4))$
$\mathbb{E}_{A_4,B_1}(f(4,1))$
$\mathbb{E}_{A_4,B_2}(f(4,2))$
$\mathbb{E}_{A_4,B_3}(f(4,3))$
$\mathbb{E}_{A_4,B_4}(f(4,4))$

热身: 混淆真值表

Alice的操作如下:

1. 写出函数 f 的真值表
2. 对每个可能的输入, 选择一个随机的**加密密钥**
3. 使用对应的密钥对输出进行加密
4. 将密文随机打乱后发送给 Bob

⁇ 以某种方式 Bob 获得 “正确的” A_x, B_y ⁇

如果Alice的输入是4, Bob的输入是2.

$\mathbb{E}_{A_3,B_4}(f(3,4))$
$\mathbb{E}_{A_4,B_3}(f(4,3))$
$\mathbb{E}_{A_3,B_3}(f(3,3))$
$\mathbb{E}_{A_2,B_3}(f(2,3))$
$\mathbb{E}_{A_4,B_2}(f(4,2))$
$\mathbb{E}_{A_2,B_4}(f(2,4))$
$\mathbb{E}_{A_4,B_4}(f(4,4))$
$\mathbb{E}_{A_1,B_4}(f(1,4))$
$\mathbb{E}_{A_2,B_2}(f(2,2))$
$\mathbb{E}_{A_1,B_2}(f(1,2))$
$\mathbb{E}_{A_2,B_1}(f(2,1))$
$\mathbb{E}_{A_1,B_3}(f(1,3))$
$\mathbb{E}_{A_4,B_1}(f(4,1))$
$\mathbb{E}_{A_3,B_1}(f(3,1))$
$\mathbb{E}_{A_1,B_1}(f(1,1))$
$\mathbb{E}_{A_3,B_2}(f(3,2))$

热身: 混淆真值表

Alice的操作如下:

1. 写出函数 f 的真值表
2. 对每个可能的输入, 选择一个随机的**加密密钥**
3. 使用对应的密钥对输出进行加密
4. 将密文随机打乱后发送给 Bob

⁇ 以某种方式 Bob 获得“正确的” A_x, B_y ⁇

通过尝试解密, Bob 只能得知 $f(x, y)$

$\mathbb{E}_{A_3,B_4}(f(3,4))$
$\mathbb{E}_{A_4,B_3}(f(4,3))$
$\mathbb{E}_{A_3,B_3}(f(3,3))$
$\mathbb{E}_{A_2,B_3}(f(2,3))$
$\mathbb{E}_{A_4,B_2}(f(4,2))$
$\mathbb{E}_{A_2,B_4}(f(2,4))$
$\mathbb{E}_{A_4,B_4}(f(4,4))$
$\mathbb{E}_{A_1,B_4}(f(1,4))$
$\mathbb{E}_{A_2,B_2}(f(2,2))$
$\mathbb{E}_{A_1,B_2}(f(1,2))$
$\mathbb{E}_{A_2,B_1}(f(2,1))$
$\mathbb{E}_{A_1,B_3}(f(1,3))$
$\mathbb{E}_{A_4,B_1}(f(4,1))$
$\mathbb{E}_{A_3,B_1}(f(3,1))$
$\mathbb{E}_{A_1,B_1}(f(1,1))$
$\mathbb{E}_{A_3,B_2}(f(3,2))$

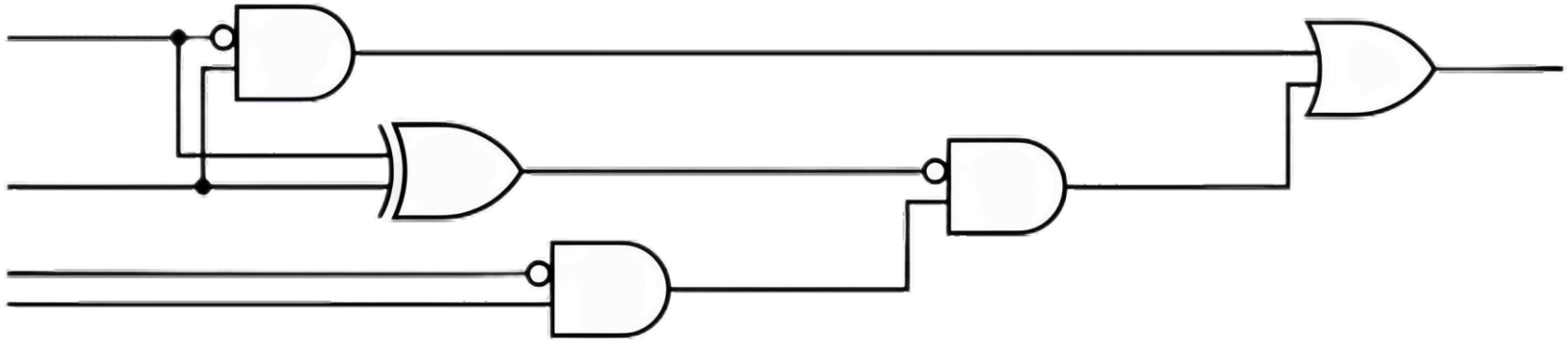
扩展热身协议

- **问题**: 成本随函数 f 的真值表大小增长!
  - 思路: 不再加密输出, 加密**指向更多混淆表的密钥**

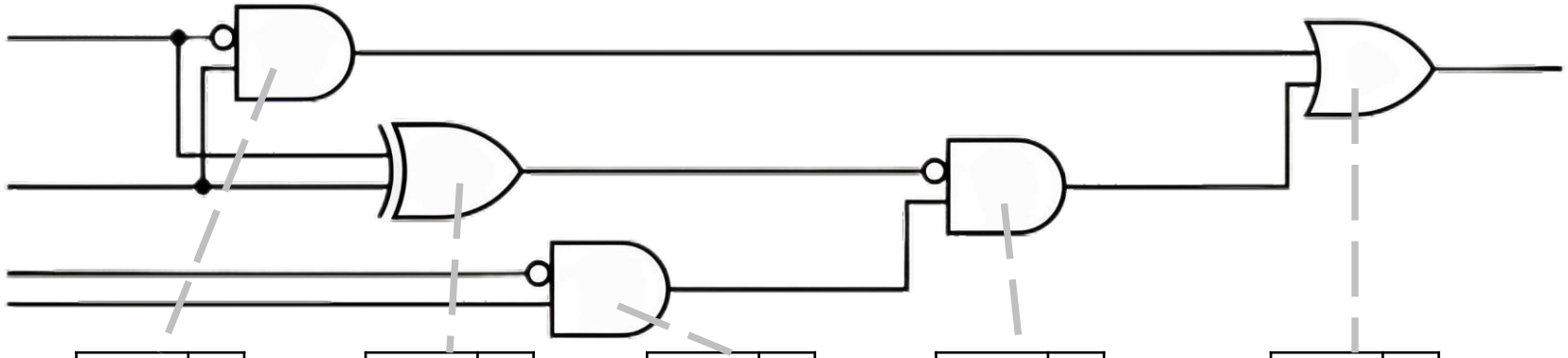
$$\boxed{\begin{array}{c} \mathbb{E}_{A_1,B_1}(C_1) \\ \mathbb{E}_{A_1,B_2}(C_1) \\ \mathbb{E}_{A_1,B_3}(C_2) \\ \mathbb{E}_{A_2,B_1}(C_1) \\ \mathbb{E}_{A_2,B_1}(C_3) \\ \vdots \end{array}} \longrightarrow \boxed{\begin{array}{c} \mathbb{E}_{C_1,D_1}(\cdots) \\ \mathbb{E}_{C_1,D_2}(\cdots) \\ \mathbb{E}_{C_1,D_3}(\cdots) \\ \mathbb{E}_{C_2,D_1}(\cdots) \\ \mathbb{E}_{C_2,D_1}(\cdots) \\ \vdots \end{array}}$$

- **问题**: Bob如何神奇地获得 “正确的” A_x, B_y?
  - 稍后讨论 (不经意传输, oblivious transfer)

混淆电路框架



混淆电路框架



0	0	0
0	1	1
1	0	0
1	1	0

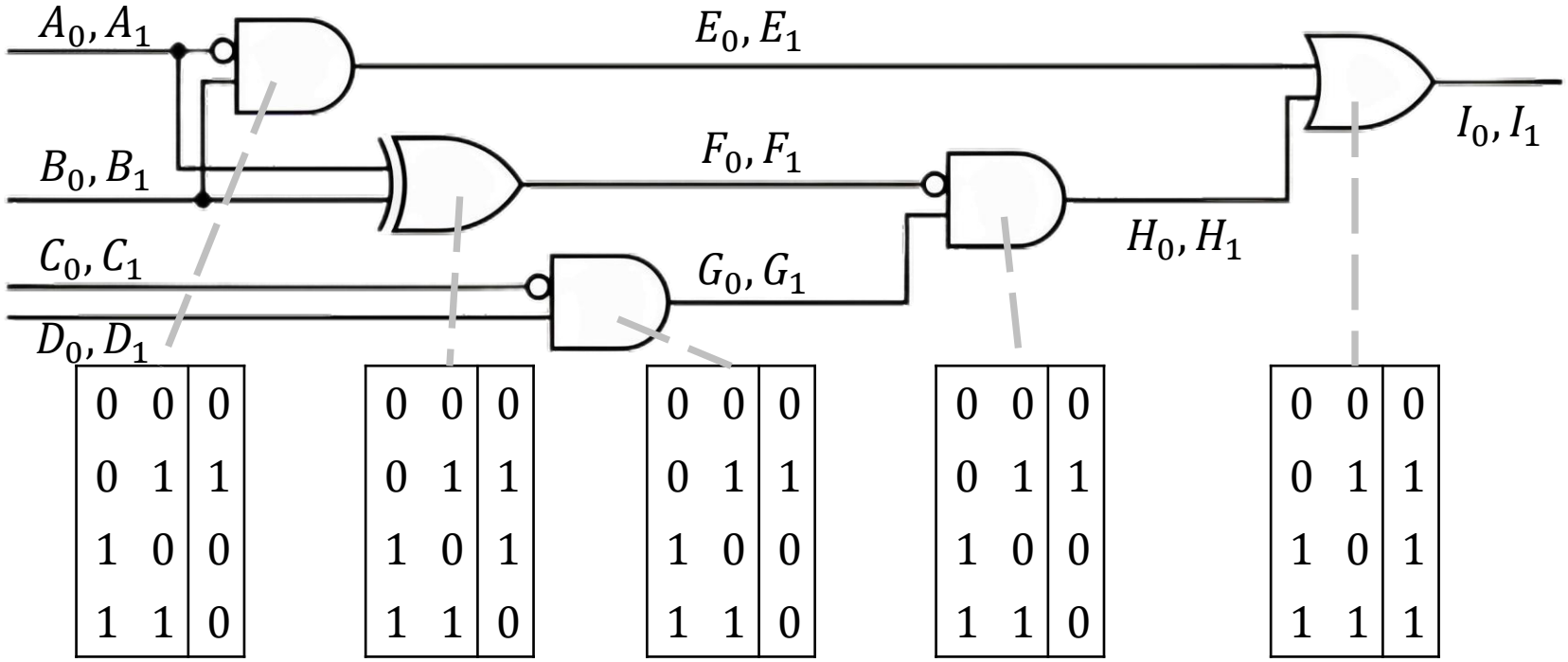
0	0	0
0	1	1
1	0	1
1	1	0

0	0	0
0	1	1
1	0	0
1	1	0

0	0	0
0	1	1
1	0	0
1	1	0

0	0	0
0	1	1
1	0	1
1	1	1

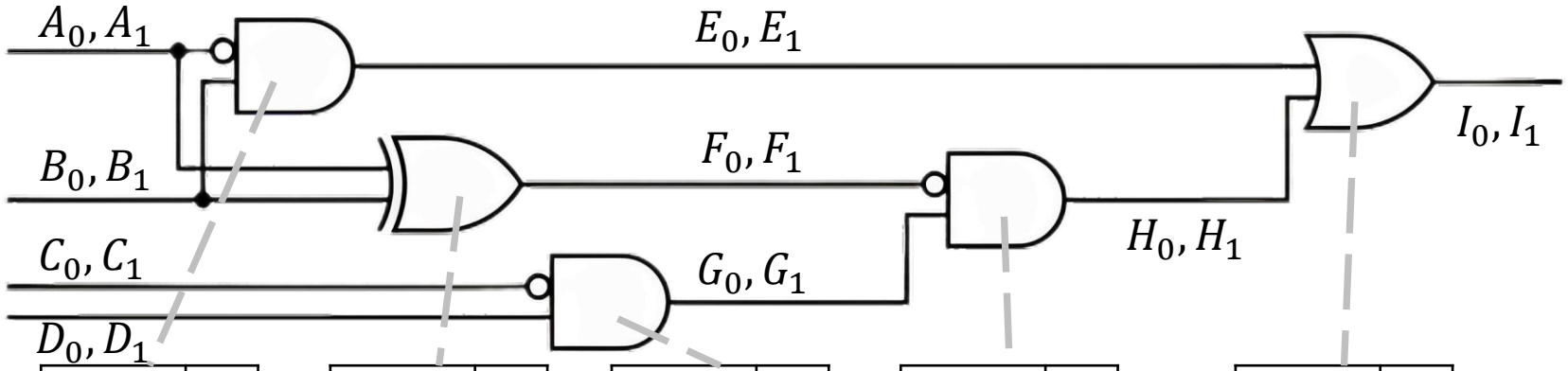
混淆电路框架



混淆一个电路:

- 为每条电线随机选择**标签** W_0, W_1

混淆电路框架



A_0	B_0	E_0
A_0	B_1	E_1
A_1	B_0	E_0
A_1	B_1	E_0

A_0	B_0	F_0
A_0	B_1	F_1
A_1	B_0	F_1
A_1	B_1	F_0

C_0	D_0	G_0
C_0	D_1	G_1
C_1	D_0	G_0
C_1	D_1	G_0

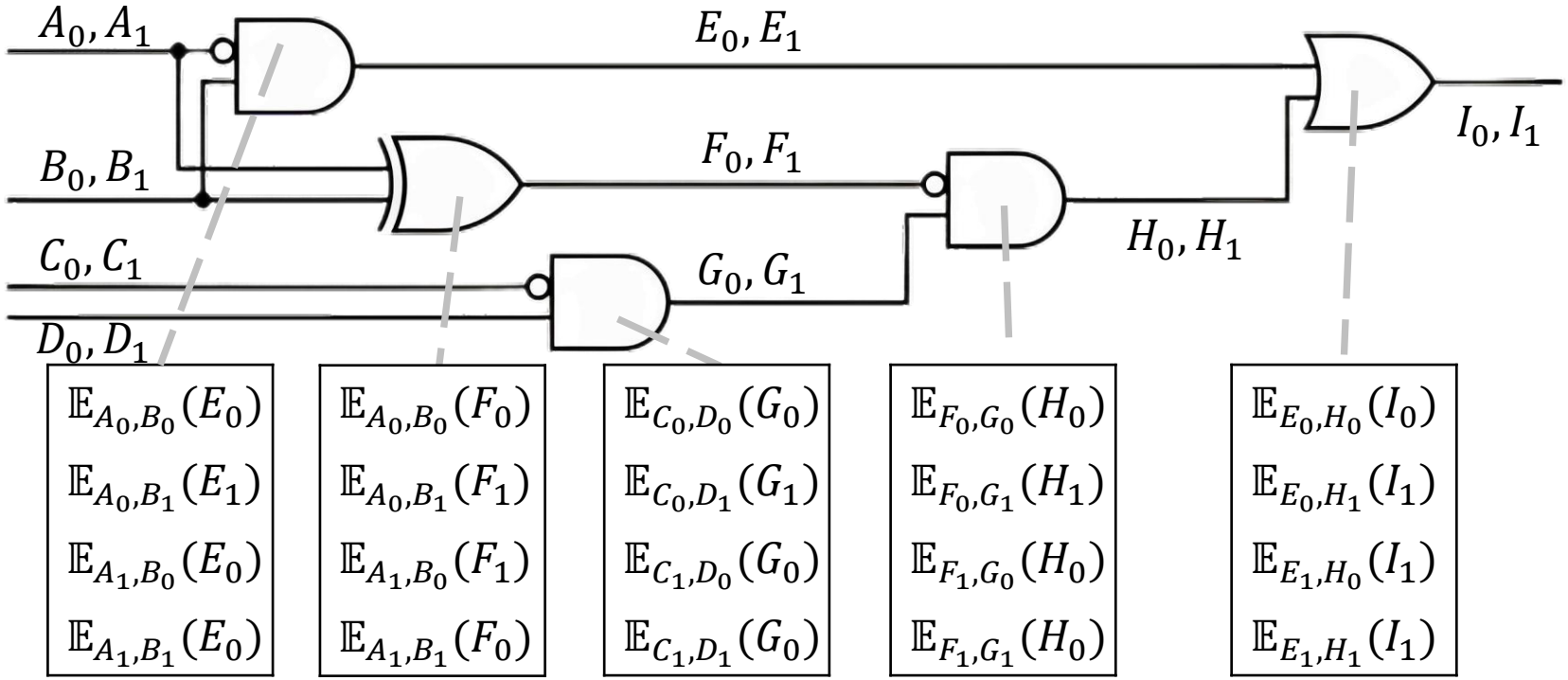
F_0	G_0	H_0
F_0	G_1	H_1
F_1	G_0	H_0
F_1	G_1	H_0

E_0	H_0	I_0
E_0	H_1	I_1
E_1	H_0	I_1
E_1	H_1	I_1

混淆一个电路:

- 为每条电线随机选择**标签** W_0, W_1

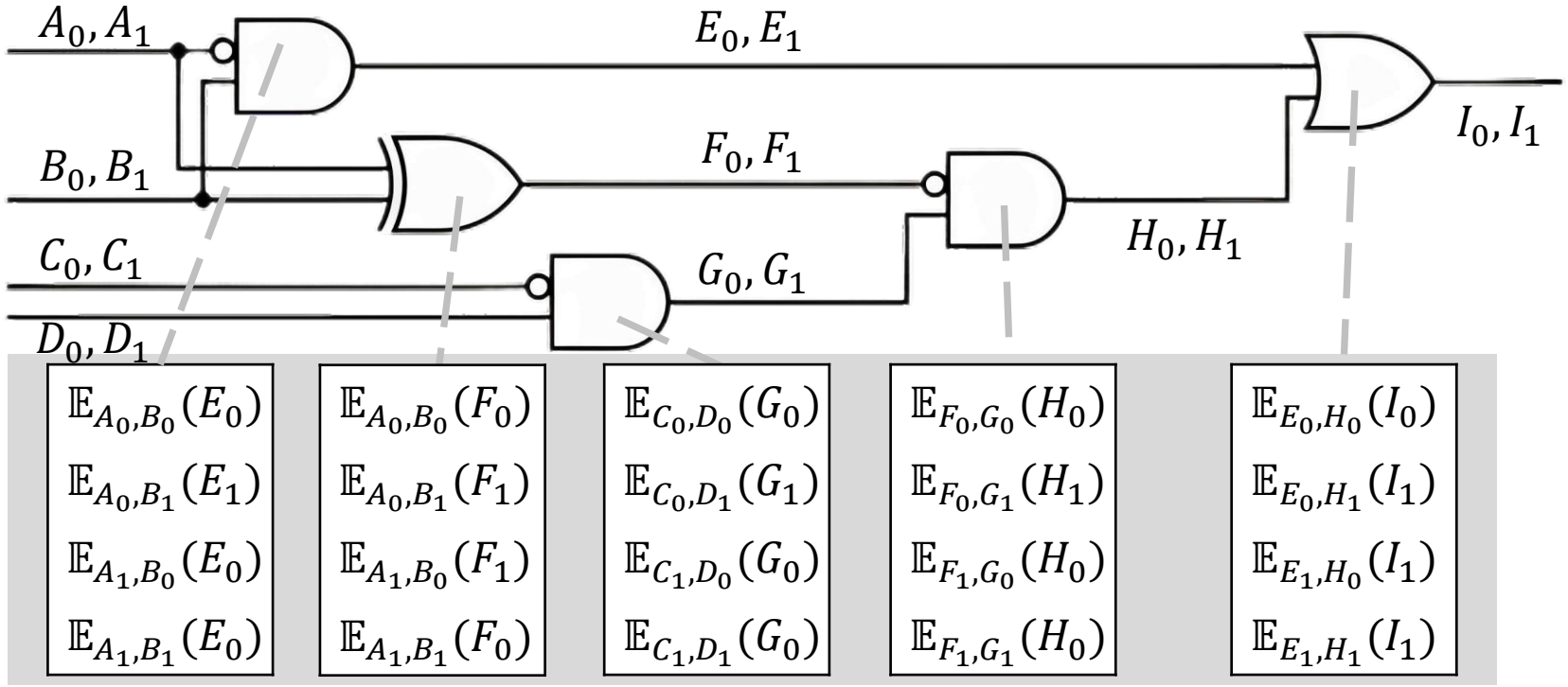
混淆电路框架



混淆一个电路:

- 为每条电线随机选择**标签** W_0, W_1
- “加密” 每个门的真值表 (打乱顺序！)

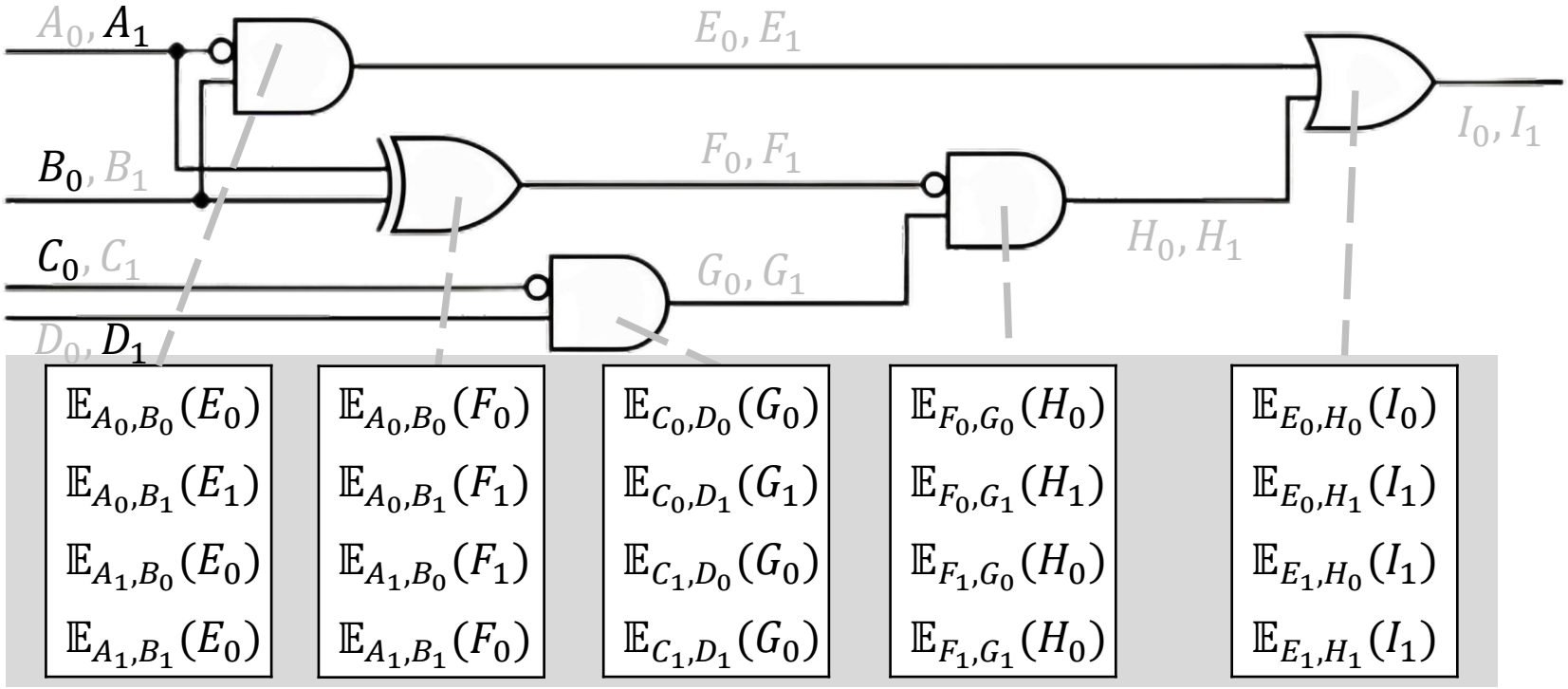
混淆电路框架



混淆一个电路:

- 为每条电线随机选择**标签** W_0, W_1
- “加密” 每个门的真值表 (打乱顺序！)
- **混淆电路** ≡ 所有加密后的门

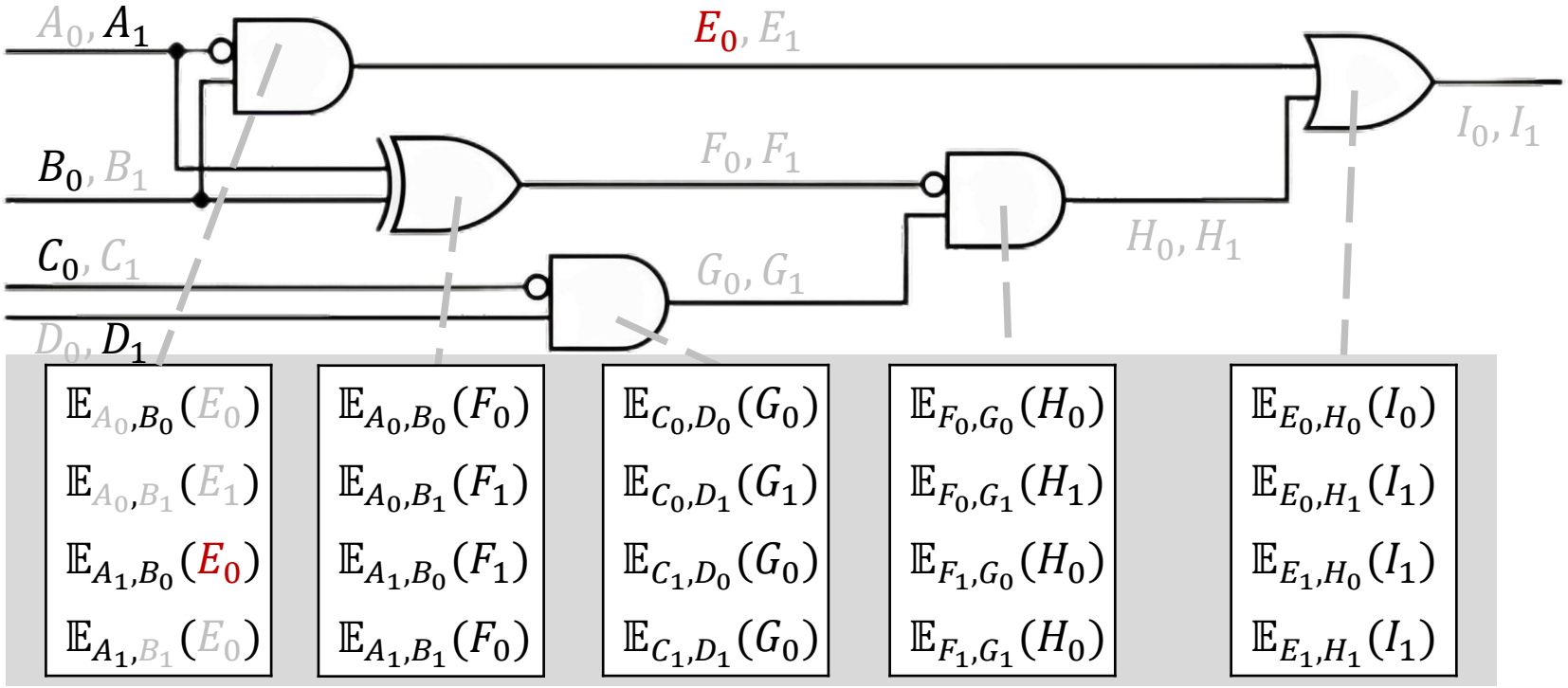
混淆电路框架



混淆一个电路:

- 为每条电线随机选择**标签** W_0, W_1
- “加密” 每个门的真值表 (打乱顺序！)
- **混淆电路** ≡ 所有加密后的门
- **混淆编码** ≡ 每条电线一个标签

混淆电路框架



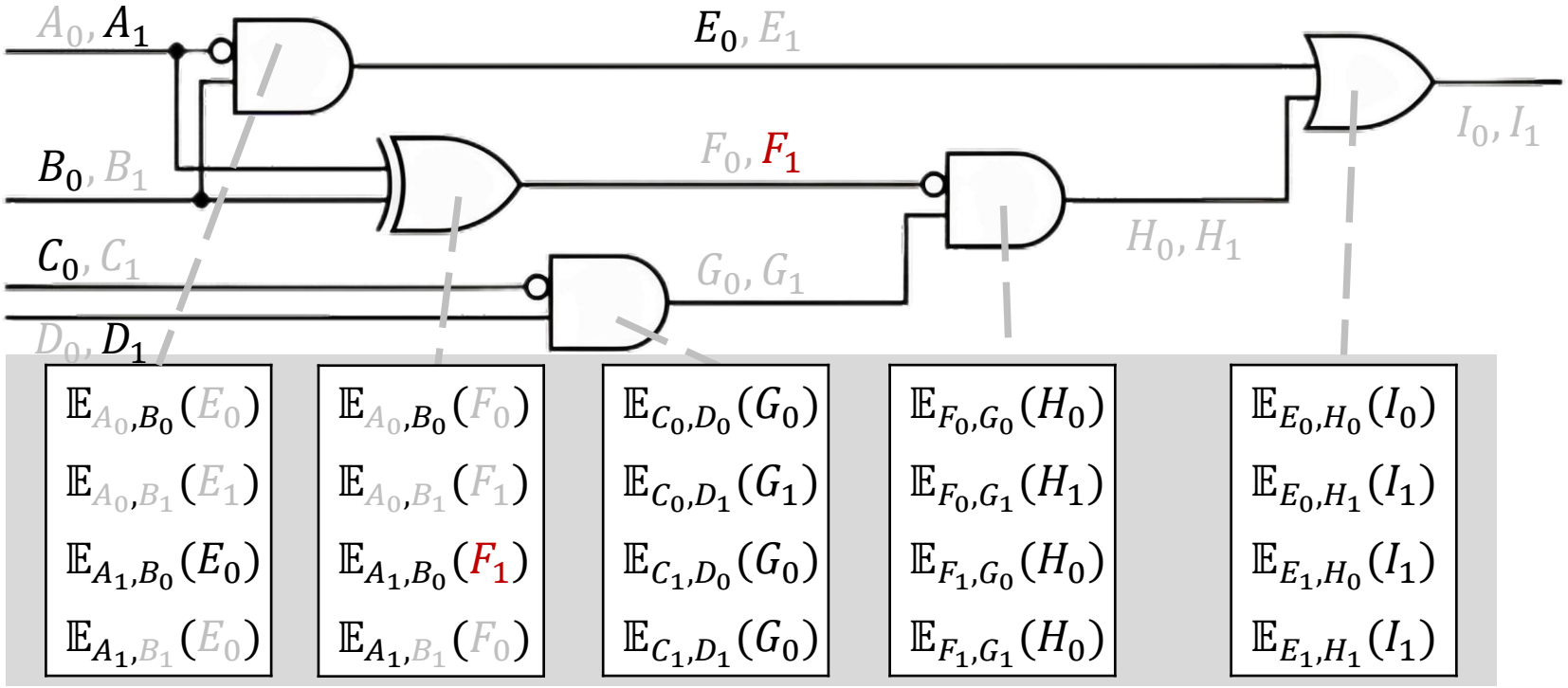
混淆一个电路:

- 为每条电线随机选择**标签** W_0, W_1
- “加密” 每个门的真值表 (打乱顺序！)
- **混淆电路** ≡ 所有加密后的门
- **混淆编码** ≡ 每条电线一个标签

混淆电路评估:

- 每个门只有一个密文可以被成功解密
- 解密结果 = 输出电线上的值

混淆电路框架



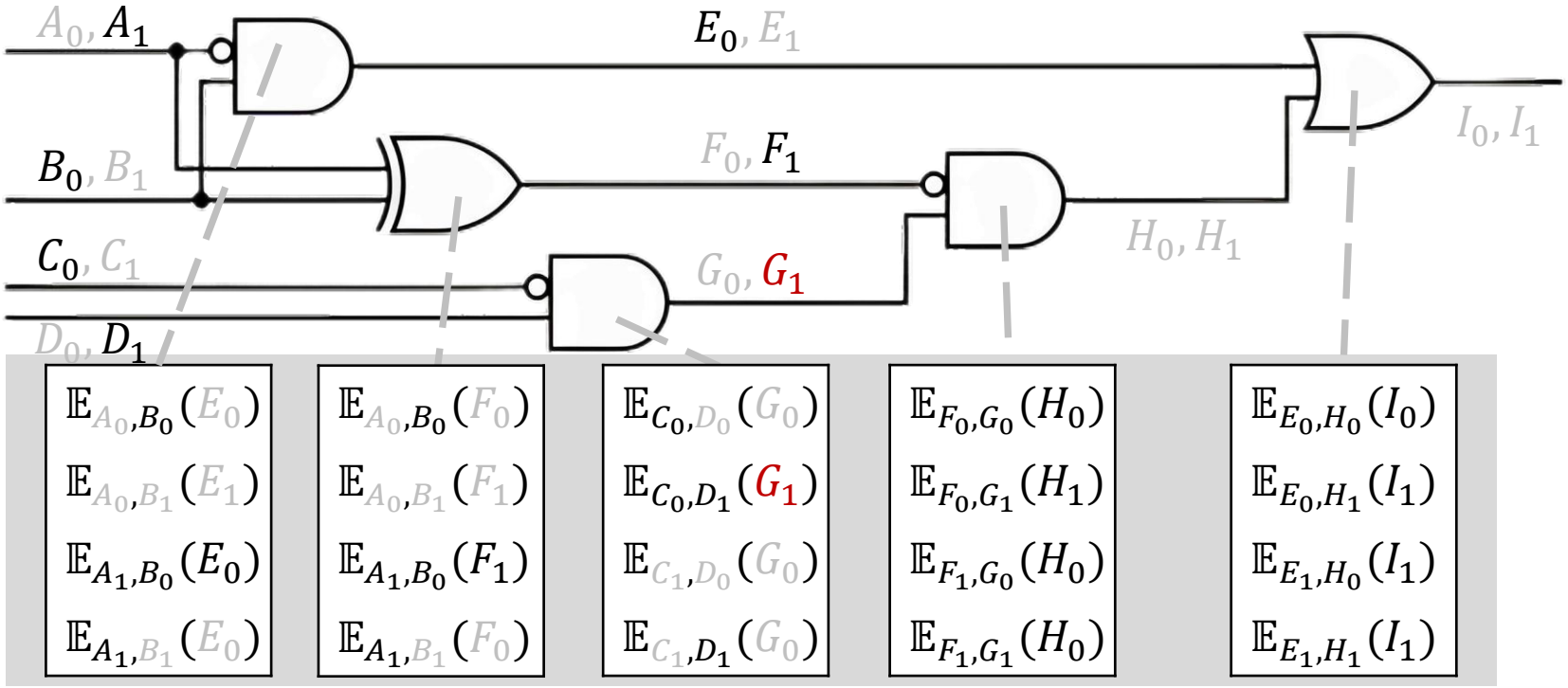
混淆一个电路:

- 为每条电线随机选择**标签** W_0, W_1
- “加密” 每个门的真值表 (打乱顺序！)
- **混淆电路** ≡ 所有加密后的门
- **混淆编码** ≡ 每条电线一个标签

混淆电路评估:

- 每个门只有一个密文可以被成功解密
- 解密结果 = 输出电线上的值

混淆电路框架



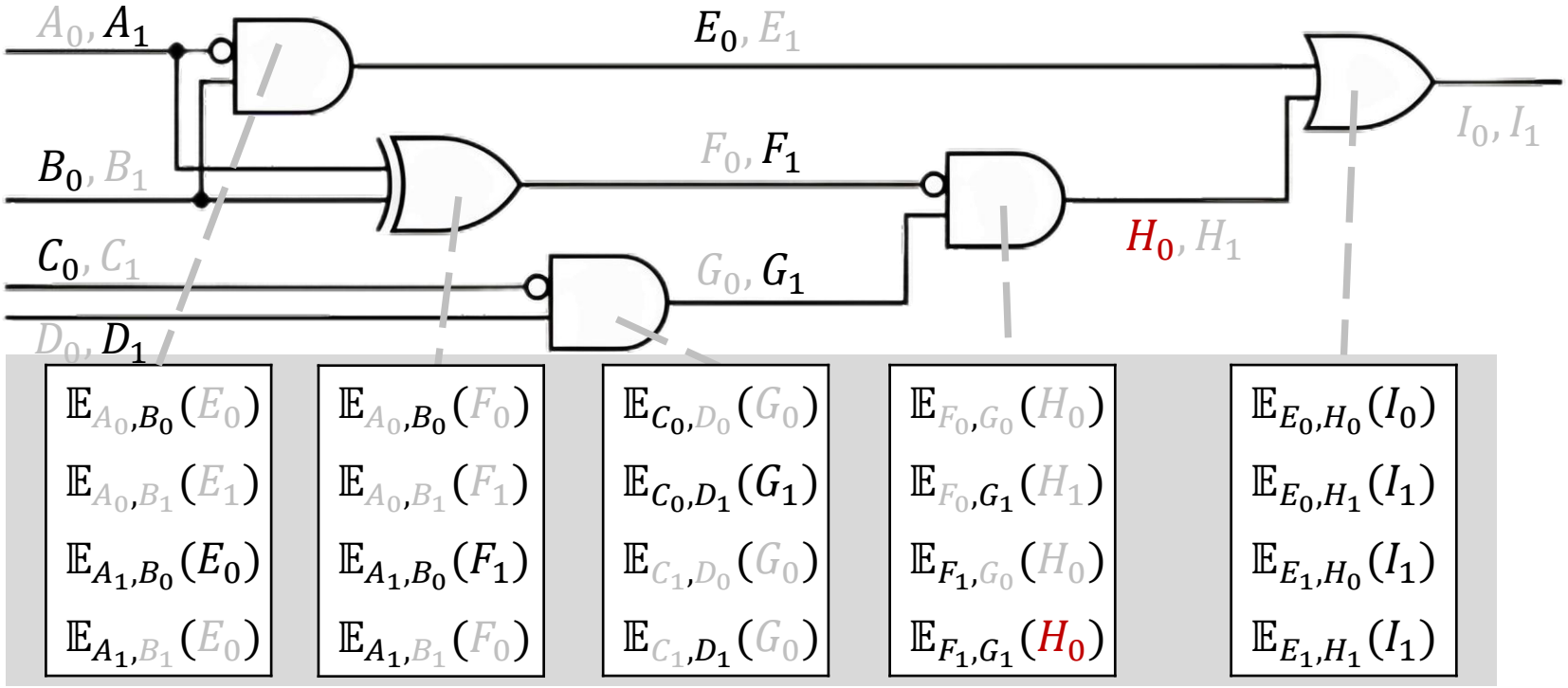
混淆一个电路:

- 为每条电线随机选择**标签** W_0, W_1
- “加密” 每个门的真值表 (打乱顺序！)
- **混淆电路** ≡ 所有加密后的门
- **混淆编码** ≡ 每条电线一个标签

混淆电路评估:

- 每个门只有一个密文可以被成功解密
- 解密结果 = 输出电线上的值

混淆电路框架



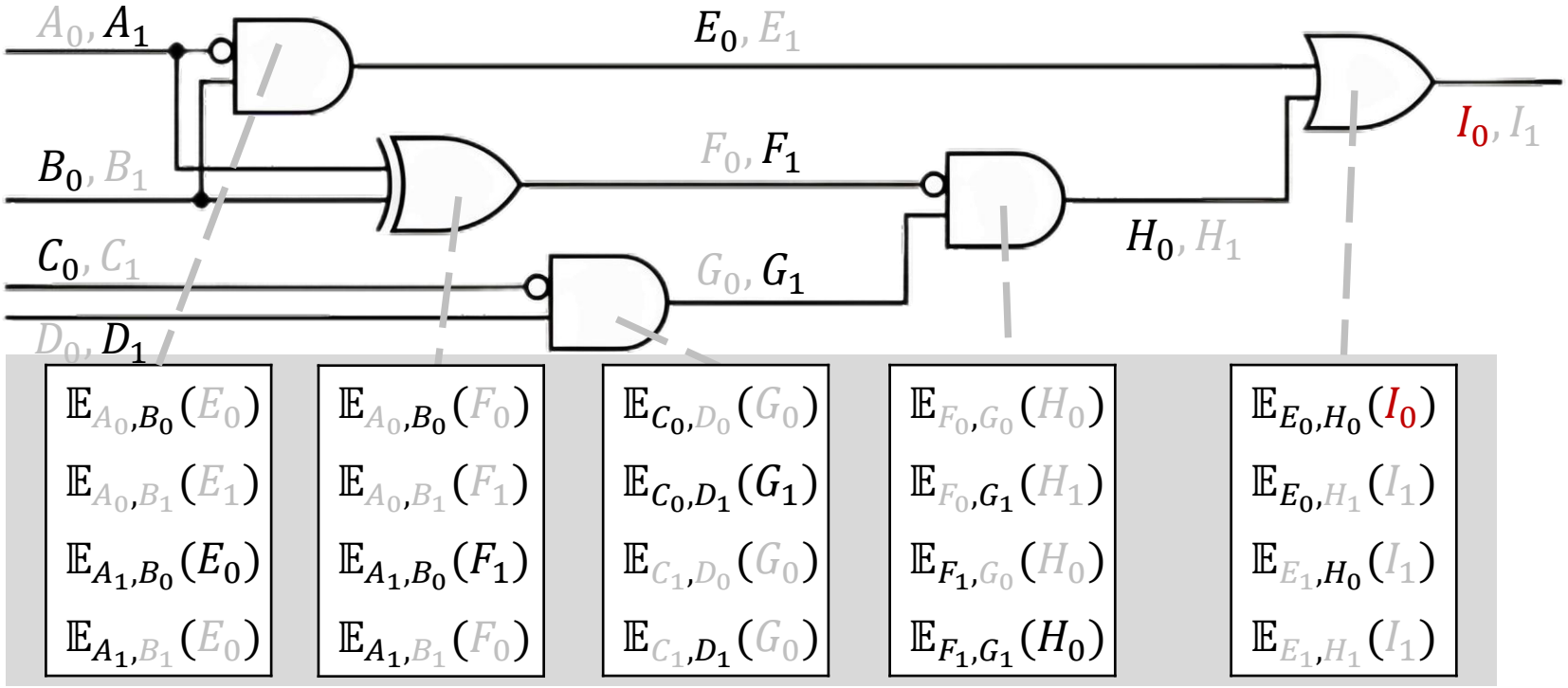
混淆一个电路:

- 为每条电线随机选择**标签** W_0, W_1
- “加密” 每个门的真值表 (打乱顺序！)
- **混淆电路** ≡ 所有加密后的门
- **混淆编码** ≡ 每条电线一个标签

混淆电路评估:

- 每个门只有一个密文可以被成功解密
- 解密结果 = 输出电线上的值

混淆电路框架



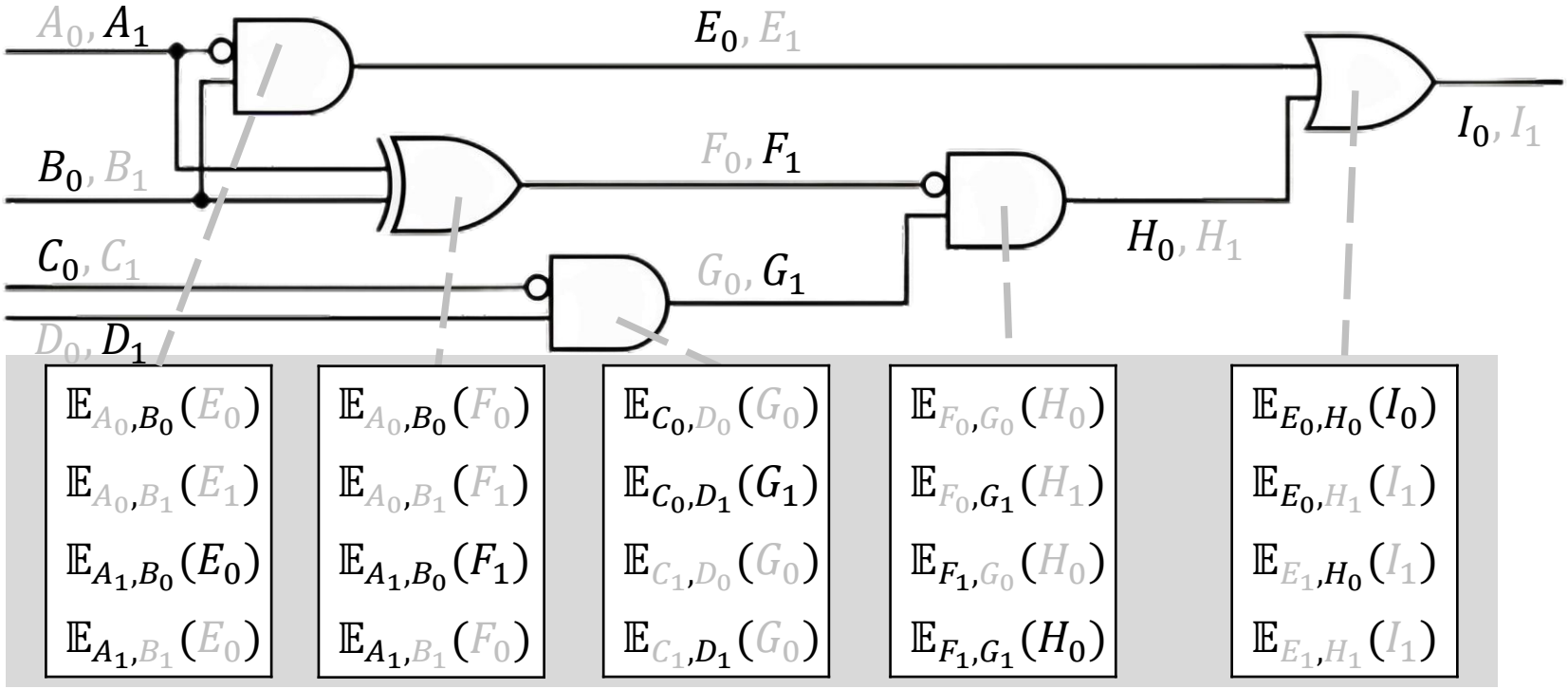
混淆一个电路:

- 为每条电线随机选择**标签** W_0, W_1
- “加密” 每个门的真值表 (打乱顺序！)
- **混淆电路** ≡ 所有加密后的门
- **混淆编码** ≡ 每条电线一个标签

混淆电路评估:

- 每个门只有一个密文可以被成功解密
- 解密结果 = 输出电线上的值

混淆电路框架



核心思路: 给定混淆电路 + 混淆输入 . . .

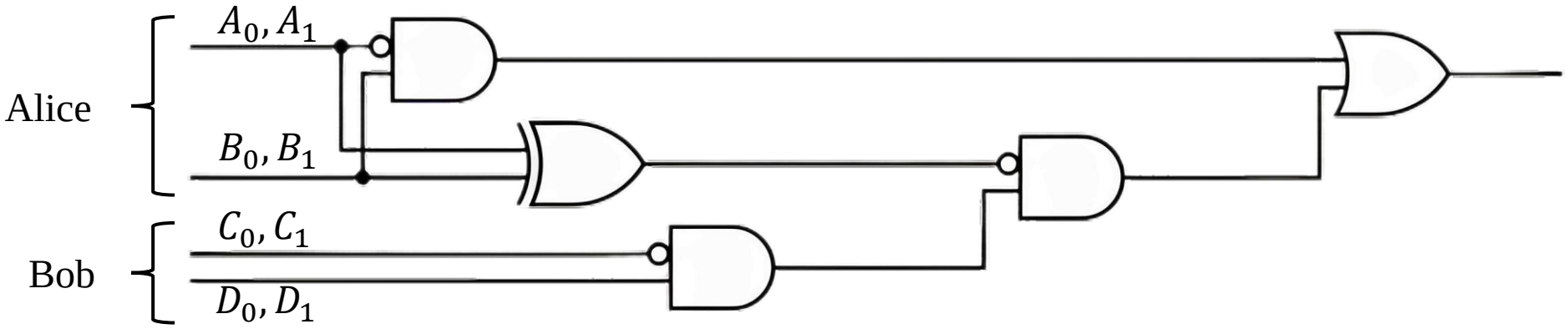
- . . . 你唯一能做的，就是 **(盲目地)** 在该输入上**计算电路**
- 每条电线只能得到一个标签: 很难猜出“互补”标签
- 看到一个标签，并不能知道电线上对应的逻辑值
- 只揭示输出线上两个标签之一，仅泄露电路的最终输出

不经意传输 (Oblivious transfer)

- 评估者 (Bob) 如何获得混淆输入?

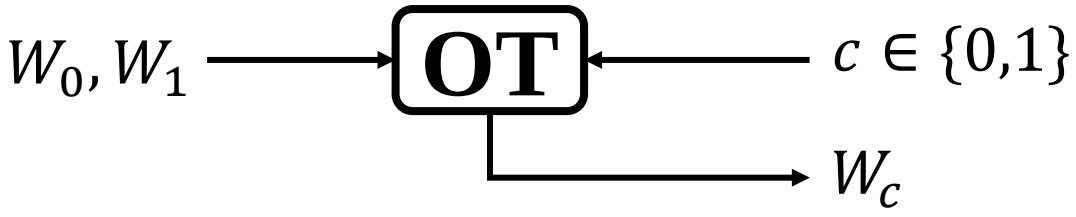
不经意传输

- 评估者(Bob) 如何获得混淆输入？



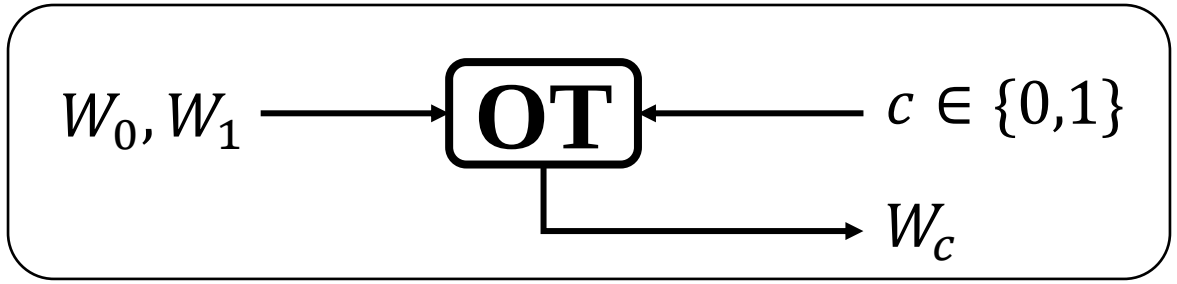
混淆者的输入: 她知道 A_0, A_1, 以及哪个是正确的 ⇒ 直接将正确的标签发送给Bob

评估者的输入: 我们需要一个“工具” (不经意传输, **oblivious transfer**):



- Bob 只能获得 W_c, 无法得知 W_{1-c}的任何信息
- Alice 不知道Bob获取了哪个标签

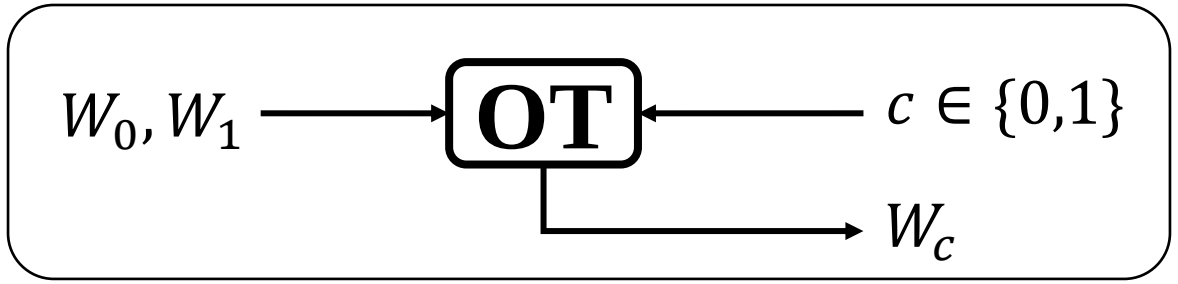
如何构造 OT?



Diffie-Hellman Key Exchange

Sender		**Receiver**
Input: (M)		Input: none
Output: none		Output: M
$a \leftarrow \mathbb{Z}_p$		$b \leftarrow \mathbb{Z}_p$
	$\xrightarrow{A = g^a}$	
	$\xleftarrow{B = g^b}$	
$k = H(B^a)$		$k = H(A^b)$
	$\xrightarrow{e \leftarrow E_k(M)}$	
		$M = D_k(e)$

如何构造 OT?



W_1, W_2

c

$a \leftarrow \mathbb{Z}_p$

$$\xrightarrow{A = g^a}$$

$b \leftarrow \mathbb{Z}_p$

If $c = 0$: $B = g^b$

If $c = 1$: $B = Ag^b$

$$\xleftarrow{B}$$

$k_0 = H(B^a)$

$k_1 = H\left(\left(\frac{B}{A}\right)^a\right)$

$$\xrightarrow{\substack{e_0 \leftarrow \mathbb{E}_{k_0}(W_0) \\ e_1 \leftarrow \mathbb{E}_{k_1}(W_1)}}$$

$k_R = H(A^b)$

$W_c = \mathbb{D}_{k_R}(e_c)$

Thank You!

密码学里的魔法

零知识证明与安全多方计算

刘逸

暨南大学网络空间安全学院